

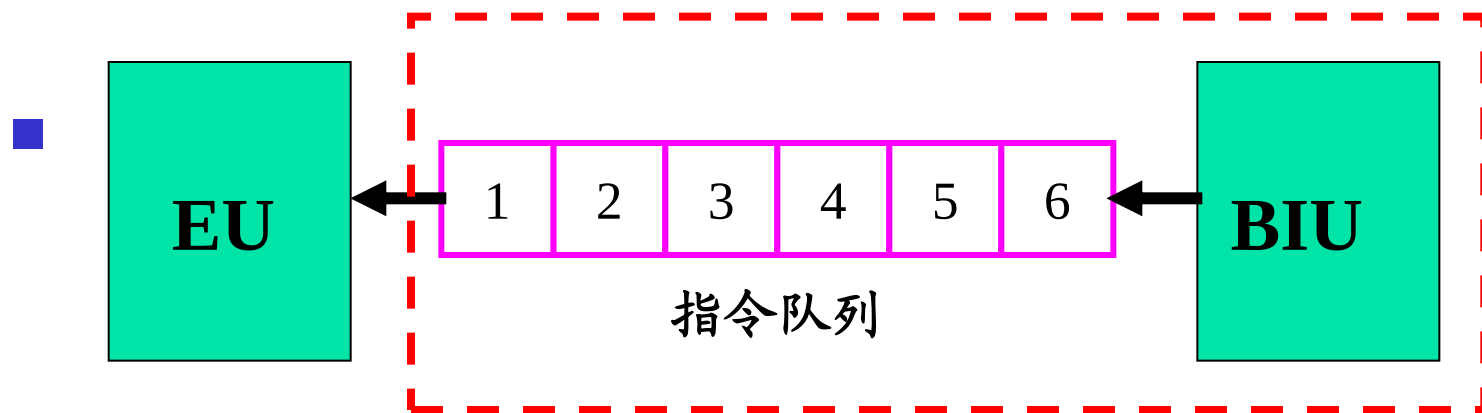
2.1 8086 CPU结构

- **8086**是**Intel**系列的**16**位微处理器，采用**HMOS**工艺制造，有**16**根数据线和**20**根地址线，封装在**40**脚双列直插组件（**DIP**）中。
- **8086**工作时，使用单一的**+5V**电源，时钟频率为**5MHz**，引脚信号与**TTL**电平兼容。
- **8086**可寻址的内存地址空间达 **2^{20}** ，即**1MB**；
- **8086**可寻址的**I/O**地址空间达 **2^{16}** ，即**64KB**；
- **Intel**公司在推出**8086**的同时一年，还推出了一款准**16**位的CPU——**8088**。
- **8088**与**8086**相比：
 - 内部结构相似，软件完全兼容。
 - **8086**的数据总线是**16**位的；
 - **8088**的数据总线是**8**位的。



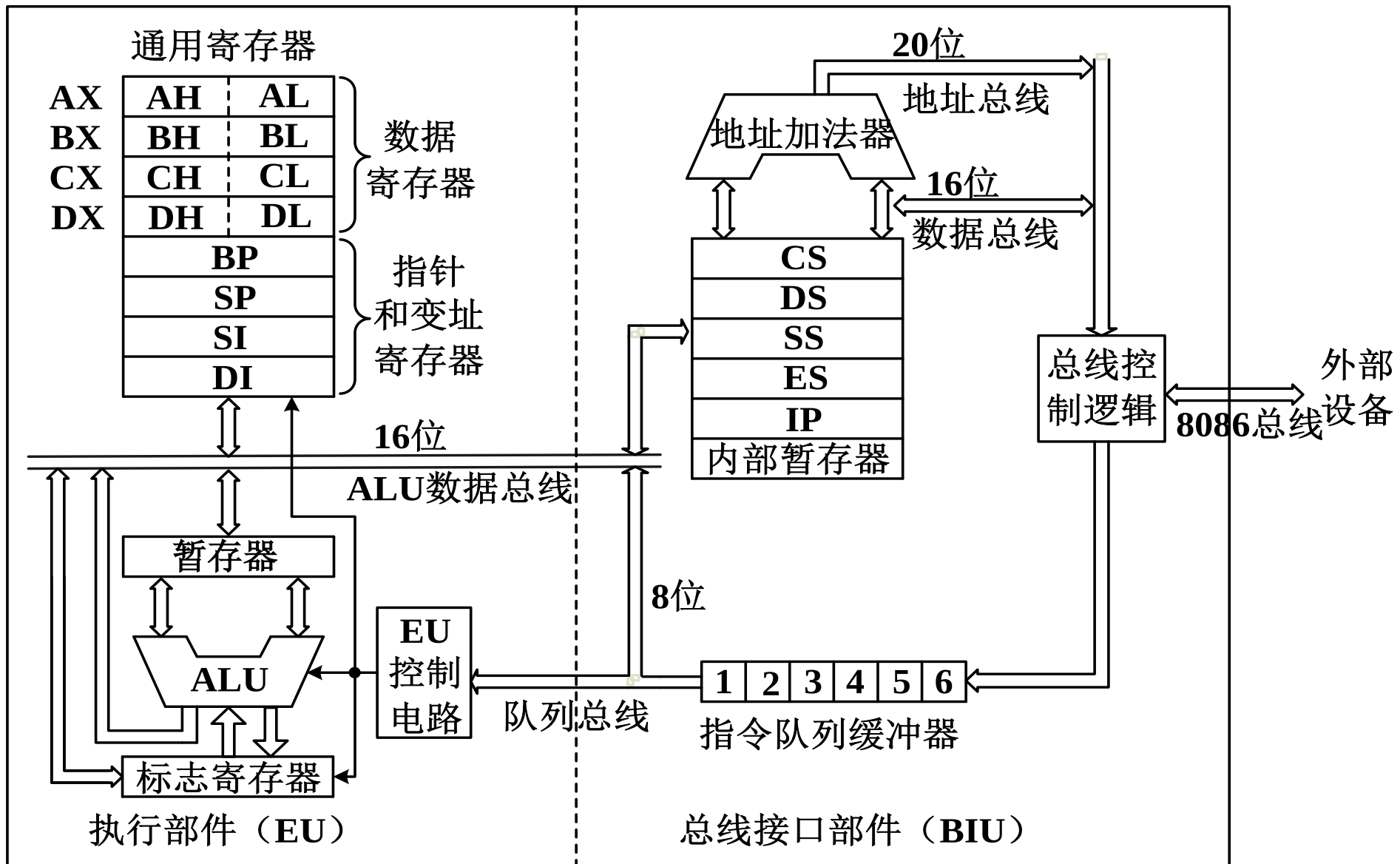
2.1.1 8086 CPU的内部结构

- **8086**由执行部件**EU**（**Execution Unit**）和总线接口部件**BIU**（**Bus Interface Unit**）两部分组成，这两个部件的操作是并行的。



- **EU**负责指令的执行；
- **BIU**负责**CPU**与存储器、**I/O**设备之间的数据传送。

8086 CPU的结构框图



1. 执行部件EU

■ EU组成:

- 算术逻辑单元（**ALU**），可完成**8位**或**16位**操作数进行算术或逻辑运算；
- **4个16位通用寄存器(AX、BX、CX、DX)**
- **4个16位专用寄存器(SI、DI、SP、BP)；**
- 标志寄存器**flags**:存放**ALU**运算结果特征
- **EU**控制器：取指令控制和时序控制电路

■ EU功能：负责指令的执行，即：

- 从总线接口部件**BIU**的指令队列取得指令，执行之后向**BIU**送回运算结果，并把运算结果的状态特征保存到标志寄存器中。

2. 总线接口部件BIU

■ **BIU组成:**

- 4个**16**位段寄存器（**DS**、**CS**、**ES**、**SS**）；
- 指令指针寄存器（**IP**）；
- **20**位的地址加法器；
- **6**字节指令队列缓冲器；
- 内部暂存器和总线控制逻辑。

■ **BIU功能:** 负责**CPU**与存储器、**I/O**设备之间的数据传送。具体包括:

- 取指令送指令队列，配合**EU**从指定的内存单元或者外设端口中取数据，将数据传送给**EU**，或者把**EU**的操作结果传送到指定的内存单元或外设端口中。

2、总线接口部件BIU

1) 段寄存器

- **8086**采用存储器地址分段的方法来解决在**16**位字长的计算机里提供**20**位地址的问题。
- 段寄存器就是专门存放段地址的寄存器，每个段寄存器的值可以确定一个段的起始地址，而各段有不同的用途。
- **8086**中有**4**个**16**位段寄存器，它们是：
 - **CS**（代码段寄存器）
 - **DS**（数据段寄存器）
 - **SS**（堆栈段寄存器）
 - **ES**（附加段寄存器）

2、总线接口部件BIU

2) 指令指针寄存器IP

- 指令指针寄存器**IP**提供下一条要取出的指令所在存储单元的**16**位偏移地址。

2、总线接口部件BIU

3) 地址加法器

- **8086CPU**采用段地址、段内偏移地址两级存储器寻址方式，由一个**20**位地址加法器根据**16**位段地址和**16**位段内偏移地址计算出**20**位的物理地址**PA**（**Physical Address**）。
- 其计算方法是：将**CPU**中的**16**位段寄存器内容左移**4**位（ $\times 16$ ）与**16**位的逻辑地址（又称偏移地址）在地址加法器内相加，得到所寻址单元的**20**位物理地址。
- 根据寻址方式的不同，偏移地址可以来自指令指针寄存器（**IP**）或其它寄存器。
- 假设（**CS**）=**FE00H**，（**IP**）=**0400H**，那么下一条要取出的指令所在内存单元的**20**位物理地址
PA=FE00H $\times$ 10H+0400H=FE400H。

2、总线接口部件BIU

4) 内部暂存器

- 用于内部数据的暂存，该部分对用户透明，在编程时可不予理会，用户无权访问。

2、总线接口部件BIU

5) 指令队列缓冲器

- **8086**有**6**字节指令队列缓冲器；
- **8088**有**4**字节指令队列缓冲器；
- 缓冲器采用“先进先出”策略，暂时存放**BIU**从存储器中预取的指令。
- 在执行指令的同时，可以从内存中取出下一条或下几条指令放到缓冲器中，一条指令执行完后，可立即执行下一条指令，从而解决了以往**CPU**取指令期间，运算器的等待问题。

2、总线接口部件BIU

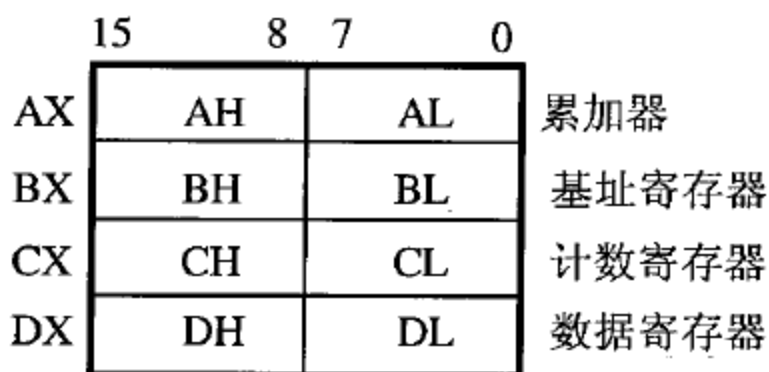
6) 总线控制逻辑

- 总线控制逻辑发出总线控制信号，实现存储器的读/写控制和**I/O**的读写控制。它将**CPU**内部总线与外部总线相连，是**CPU**与外部电路进行数据交换的路径。
- 总线控制逻辑控制**8086**通过**20**条引脚线分时传送**20**位地址线、**16**位数据和**4**位状态信息。

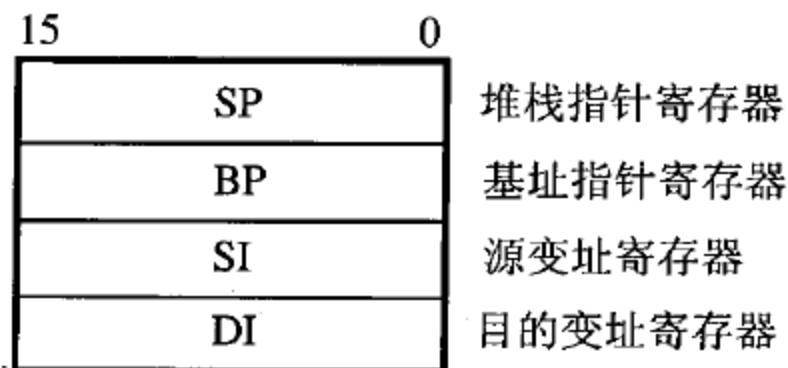
2.1.2 寄存器结构

- **寄存器**是**CPU**内部用来存放地址、数据和状态标志的部件。
- **8086**有**4组16位**寄存器：
 - 通用寄存器（数据寄存器）**4**个，可分拆成**8**个**8**位寄存器；
 - 指针和变址寄存器；
 - 段寄存器；
 - 指令指针和标志寄存器。

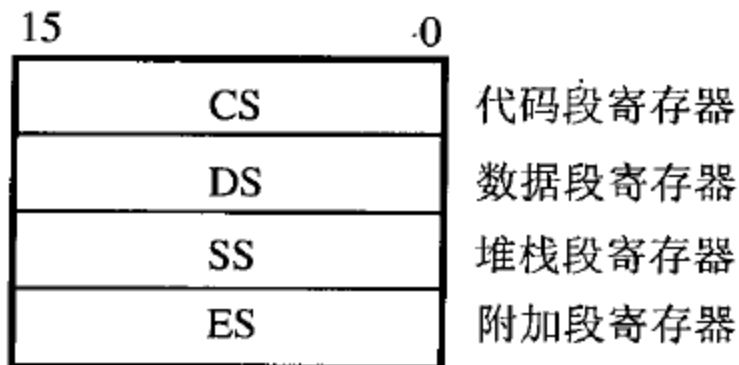
8086 CPU的寄存器组



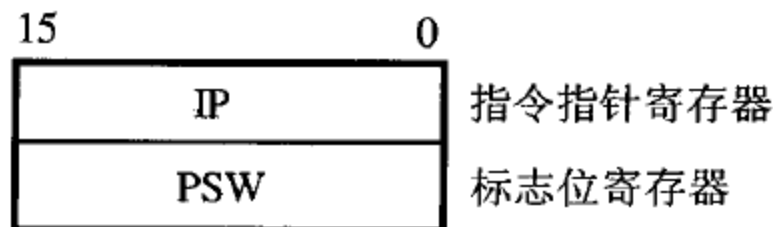
(a) 通用寄存器组



(b) 指针和变址寄存器



(c) 段寄存器



(d) 指令指针和标志位寄存器

图 2.2 8086 CPU 寄存器组

1. 通用寄存器**AX、BX、CX、DX**

- 通用寄存器主要用来存放操作数或中间结果，以减少访问存储器的次数。有：
 - 4个**16**位的寄存器：**AX、BX、CX、DX**；
 - 或分成**8**个**8**位的寄存器：
 - **AH、AL、BH、BL、CH、CL、DH、DL**；
 - 这些寄存器均可独立使用。
- 多数情况下，这些数据寄存器是用于算术运算或逻辑运算指令中，以进行算术逻辑运算。在有些指令中，它们则有特定的用途。
- 这些寄存器的用法见下表所示。

1. 通用寄存器AX、BX、CX、DX

寄存器	一般用法	隐含用法
AX	16位累加器	①字节乘法中保存积；字乘法中隐含提供一个乘数，并保存积的低16位； ②字节除法中隐含提供被除数；字除法中隐含提供被除数的低16位，并保存商； ③CBW指令中隐含作为目标操作数； ④CWD指令中隐含作为源操作数和目标操作数的低16位； ⑤I/O指令中，保存16位输入/输出数据。
AL	AX的低8位	①字节乘法中隐含提供一个乘数，并保存积的低8位；字节除法中隐含提供被除数的低8位并保存商； ②CBW指令中隐含作为源操作数； ③XLAT指令中隐含提供表格首地址偏移量； ④I/O指令中，保存8位输入/输出数据。
AH	AX的高8位	①字节乘法中隐含提供一个乘数； ②字节除法中隐含保存余数； ③DOS和BIOS功能调用中存放功能号。

1. 通用寄存器AX、BX、CX、DX

寄存器	一般用法	隐含用法
BX	基址寄存器，常用作地址寄存器	XLAT指令中提供被查表格中源操作数的间接地址。
CX	16位计数器	①循环指令中的循环次数计数器； ②串操作指令中串长计数器。
CL	CX的低8位	移位或循环移位指令中提供移位的次数。
DX	16位数据寄存器	①字乘法中隐含保存积的高16位； ②字除法中隐含提供被除数的高16位，并保存积的高16位； ③CWD指令中隐含作为目标操作数的高16位； ④在间接寻址的I/O指令中，提供端口地址。

2. 指针和变址寄存器SP、BP、SI、DI

- **EU**中有**2**个地址指针寄存器**SP**、**BP**和**2**个变址寄存器**SI**、**DI**。用法如下表所示：

寄存器	一般用法	隐含用法
SP (堆栈指针寄存器)	保存堆栈栈顶偏移地址，与SS配合来确定堆栈在内存中的位置。	压栈、出栈操作中隐含指示栈顶
BP (基址指针寄存器)	①保存16位数据 ②保存堆栈段内存储单元的偏移地址。	
SI (源变址寄存器)	①保存16位数据 ②保存数据段内存储单元的偏移地址。	串操作指令中，隐含与DS配合，确定源串在内存中的位置。
DI (目的变址寄存器)	①保存16位数据 ②保存数据段内存储单元的偏移地址。	串操作指令中，隐含与ES配合，确定目标串在内存中的位置。

3. 段寄存器

- **8086**内部设置了**4个16位**的段寄存器，用于存放当前程序所用的各段的起始地址的高**16位**：
- **CS（Code Segment）**，称为代码段寄存器，存放当前执行的程序所在段的起始地址。其值乘以**16**再加上**IP**的值，就形成了下一条要取出指令所在的内存单元的物理地址。
- **DS（Data Segment）**，称为数据段寄存器，存放当前数据段的起始地址。其值乘以**16**再加上指令中存储器寻址方式指定的偏移地址，就形成了要进行读/写的数据段中指定内存单元的物理地址。

3、段寄存器

- **SS (Stack Segment)**，称为堆栈段寄存器，存放当前堆栈段的起始地址。堆栈是按照“后进先出”原则组织的一个特殊内存区域。堆栈操作数的地址由**SS**的值乘以**16**再加上**SP**的值形成。
- **ES (Extra Segment)**，称为附加段寄存器，存放当前附加段的起始地址。附加段是附加的数据段，也用于数据的保存，另外，串操作指令将附加段作为其目标操作数的存放区域。

4. 16位指令指针寄存器IP

- 指令指针寄存器**IP**（**Instruction Pointer**）存放当前代码段中的偏移地址，它与**CS**联用，可以形成下一条要取出指令的物理地址。
- 程序不能直接对**IP**进行存取，但能在程序运行中被自动修改。
 - 例如，控制器取到要执行的指令后，会立刻修改**IP**值，使之指向下一条指令的首地址；
 - 转移、调用、返回等指令执行，就是通过修改**IP**的值来控制指令序列的执行流程的。

5. 标志寄存器flags

- **flags**是一个**16**位的寄存器。**9**个位用作标志位：
 - **状态标志位**有**6**个，记录程序中运行结果的状态信息，是根据指令的运行结果由**CPU**自动设置的。这些状态信息通常作为后续转移指令的转移控制条件，所以也称为条件码。
 - **控制标志位**有**3**个，可以编程设置，用于控制处理器执行指令的方式。控制标志设置之后，可对后面的操作产生控制作用。
- 未标明的位在**8086**中不用。

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF

5、标志寄存器flags —进位标志

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF



进位标志：指令执行后，如果运算结果在最高位上产生了一个进位或借位，则 $CF=1$ ；否则， $CF=0$ 。

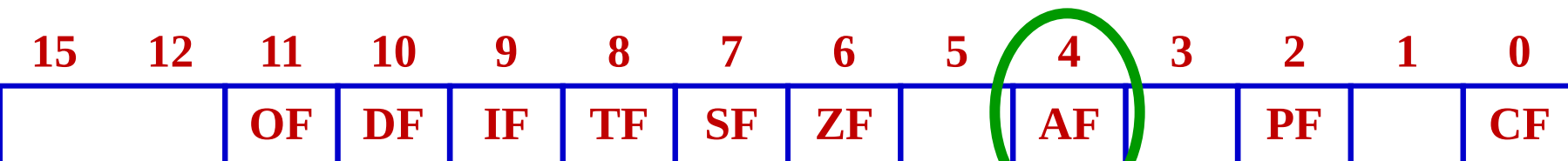
5、标志寄存器flags — 奇偶标志

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF



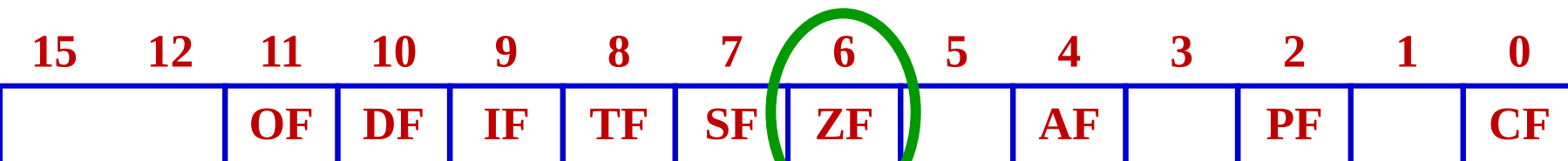
奇偶标志：如果运算结果低八位1的个数为偶数，则 $PF=1$ ；否则， $PF=0$ 。

5、标志寄存器flags—辅助进位标志



辅助进位标志：如果运算结果低4位产生了进位，则 $AF=1$ ；否则， $AF=0$ 。

5、标志寄存器flags—零标志



零标志：如果运算结果为0，则ZF=1；否则，ZF=0。

5、标志寄存器flags—符号标志

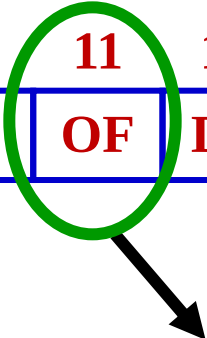
15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF



符号标志：如果运算结果为正数，
则 $SF=0$ ；否则， $SF=1$ 。

5、标志寄存器flags—溢出标志

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF

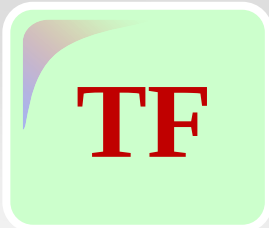


OF

溢出标志：如果运算过程产生了溢出，则OF=1；否则，OF=0。

5、标志寄存器flags—单步标志

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF

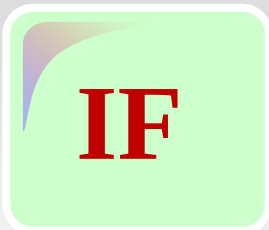


单步标志：

TF=1，处理器按单步执行指令；
TF=0，处理器正常工作。

5、标志寄存器flags—中断允许标志

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF



中断允许标志：

IF=1，允许可屏蔽中断；

IF=0，不允许可屏蔽中断。

5、标志寄存器flags—方向标志

15	12	11	10	9	8	7	6	5	4	3	2	1	0
		OF	DF	IF	TF	SF	ZF		AF		PF		CF

DF

方向标志（用于串操作指令）：
DF=1，存储器地址会自动减值；
DF=0，存储器地址会自动增值。

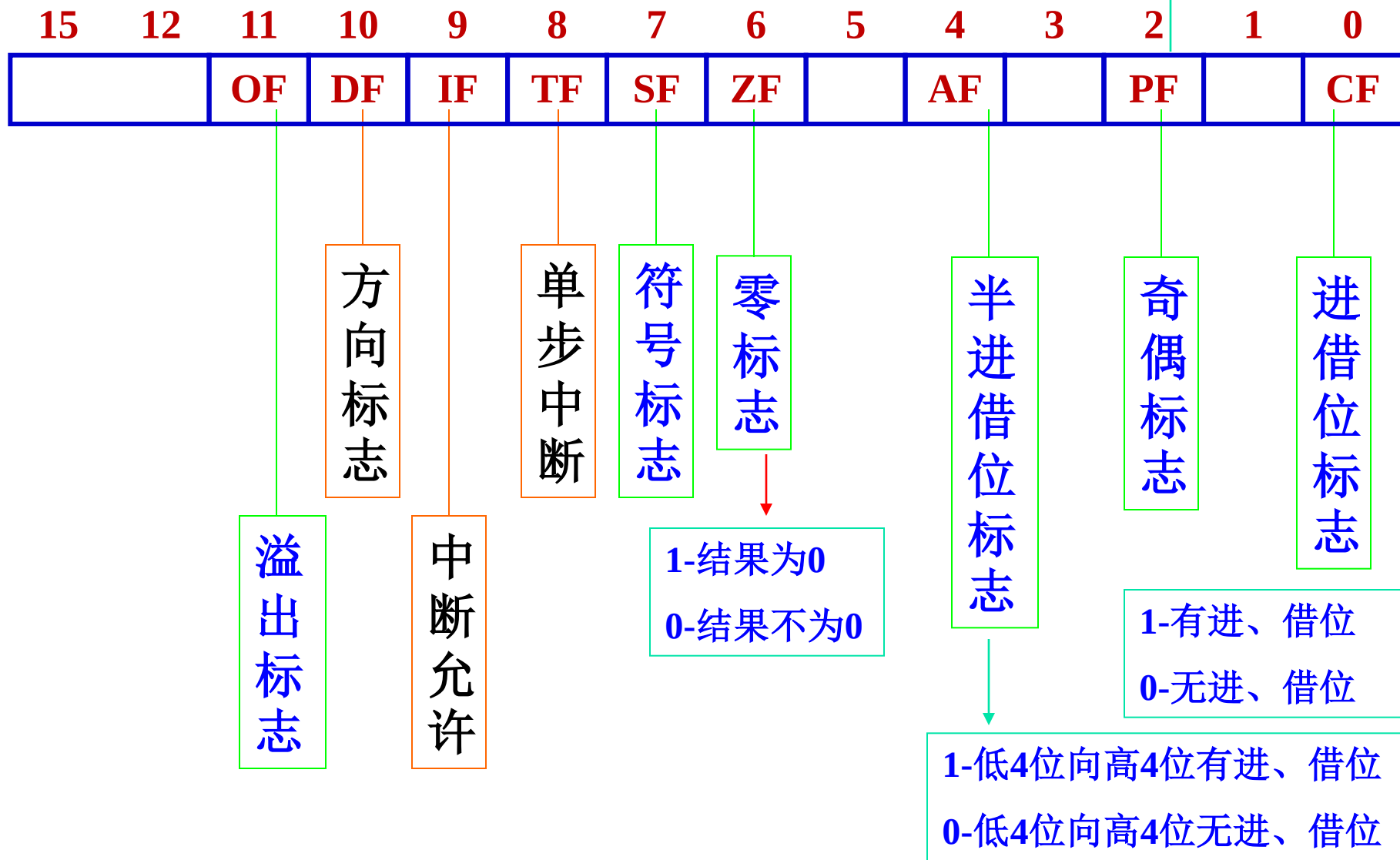
5、标志寄存器**flags**—标志位符号值

- 状态标志的状态表示在PC 中，可由调试程序（DEBUG）显示出来。
- 对应符号如下表所示：

标志位名	为 1 对应符	为 0 对应符
OF	OV	NV
DF	DN	UP
IF	EI	DI
SF	NG	PL
ZF	ZR	NZ
AF	AC	NA
PF	PE	PO
CF	CY	NC

16位的flags，只用9位

其中：状态标志6位，控制标志3位



例：将5394H与-777FH相加，并说明其标志位的状态。

【解】先求-777FH的补码：

原码： 1111 0111 0111 1111

反码： 1000 1000 1000 0000

补码： 1000 1000 1000 0001

再做加法运算： 0101 0011 1001 0100

+ 1000 1000 1000 0001

1101 1100 0001 0101

结果标志位为： CF=0(NC)、 PF=0(PO)、 AF=0(NA)

ZF=0(NZ)、 SF=1(NG)、 OF=0(NO)

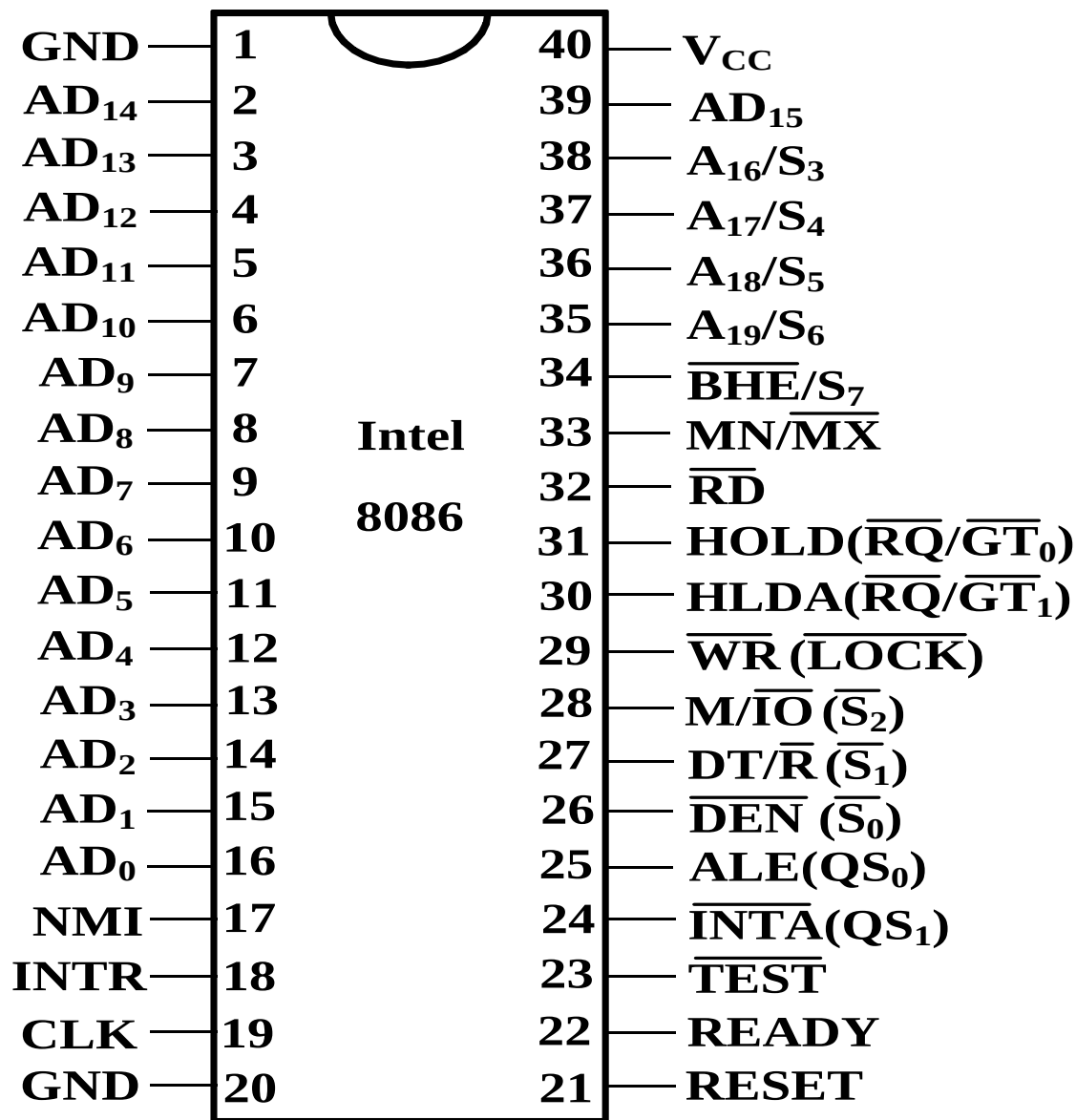
2.2 8086 CPU引脚及其功能

- 根据**8086CPU**的功能，应该包括**20**位地址线，**16**位数据线以及若干个控制线。
- 限于当时集成电路工艺条件，采用分时复用方式，只引出**40**个引脚。
- 有两种工作模式：**最小模式** 和 **最大模式**
- **24 ~ 31**的**8**条引脚在两种模式下有不同的用法

2.2 8086 CPU的引脚及其功能

- 当把**8086CPU**与存储器、外设构成一个计算机系统时，根据所连的存储器和外设的规模，**8086CPU**具有两种不同的工作模式来适应不同的应用场合：
 - 最小模式
 - 最大模式
- **8086**的工作模式由硬件设计决定：
 - $\overline{\text{MN}}/\overline{\text{MX}}$ 引脚连电源（+5V），则**8086**处在最小模式；
 - $\overline{\text{MN}}/\overline{\text{MX}}$ 引脚接地，则**8086**处在最大模式。

DIP封装 引脚图



最小模式

- 最小模式也称为单处理器模式，是指系统中只有一片**8086**微处理器，所连的存储器容量不大、片子不多，所要连的**I/O**端口也不多，系统的控制总线就直接由**CPU**的控制线供给，从而使得系统中的总线控制电路被减到最少。
- 最小模式适用于较小规模的系统。

最大模式

- 最大模式是相对于最小模式而言的，适用于中、大型规模的系统。
 - 在最大模式的系统中有多多个微处理器，其中一个主处理器**8086**，其他的处理器称为协处理器，承担某方面专门的工作。和**8086**配合的协处理器有数值运算协处理器**8087**，和输入/输出协处理器**8089**。
 - **8086**通过一个总线控制器**8288**来形成各种总线周期，控制信号由**8288**供给。

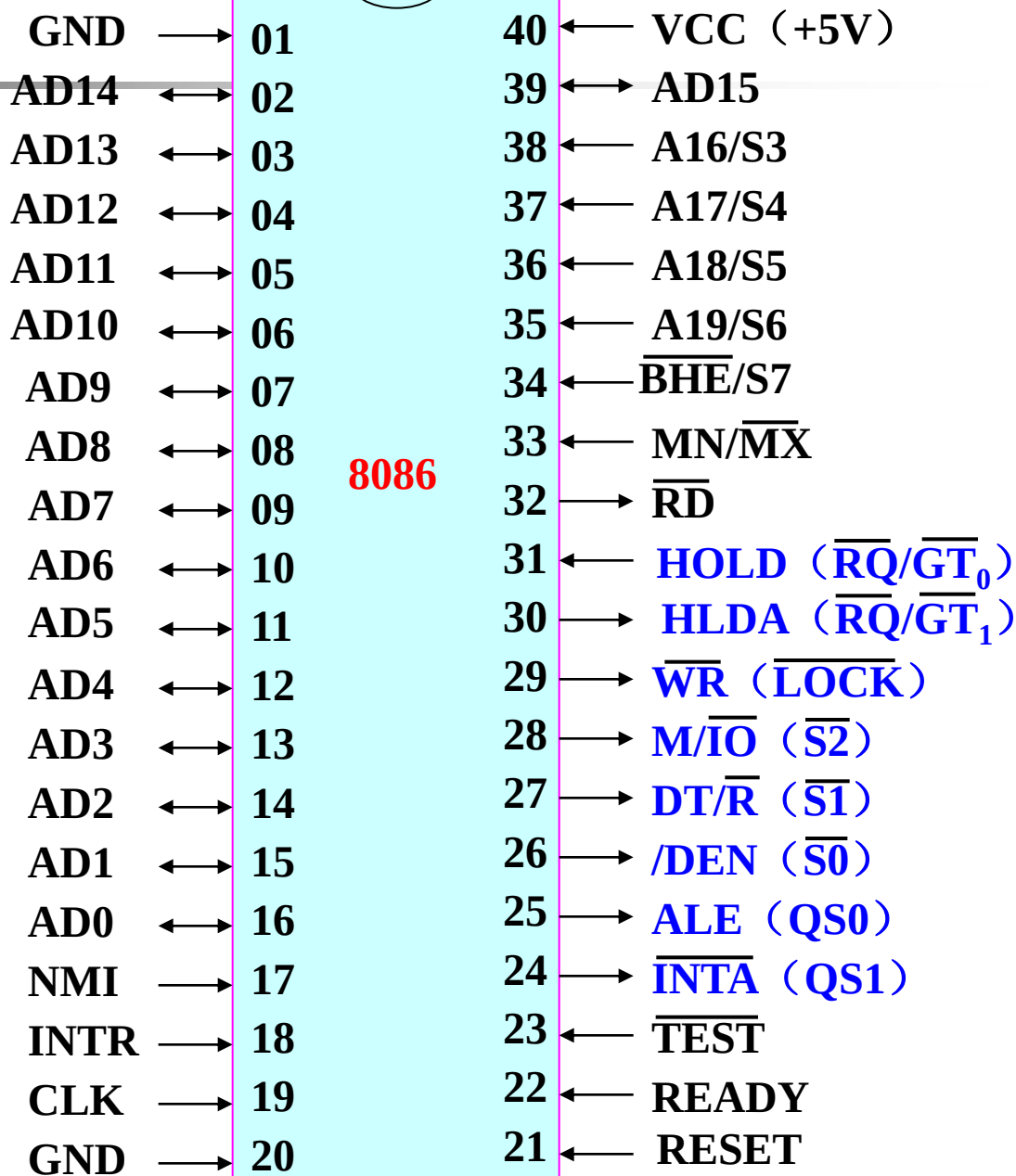
2.2.1 8086 CPU的引脚定义

描述方面:

- 引脚的功能
- 信号的有效电平
- 信号的流向
- 引脚的复用
- 三态能力

引脚及信号 流向图

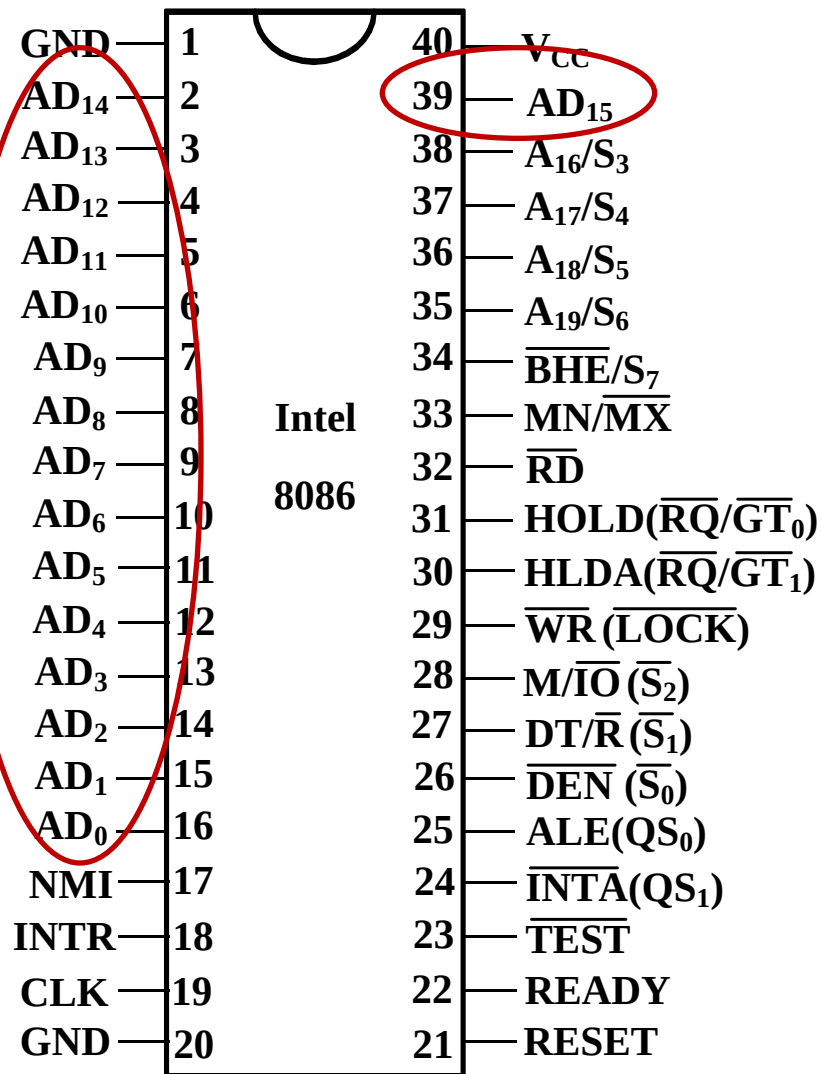
- 数据总线为**16**位
- 地址总线为**20**位
- **40**条引脚
- 部分引脚分时复用



1、两种工作模式的公共引脚

AD₀~AD₁₅

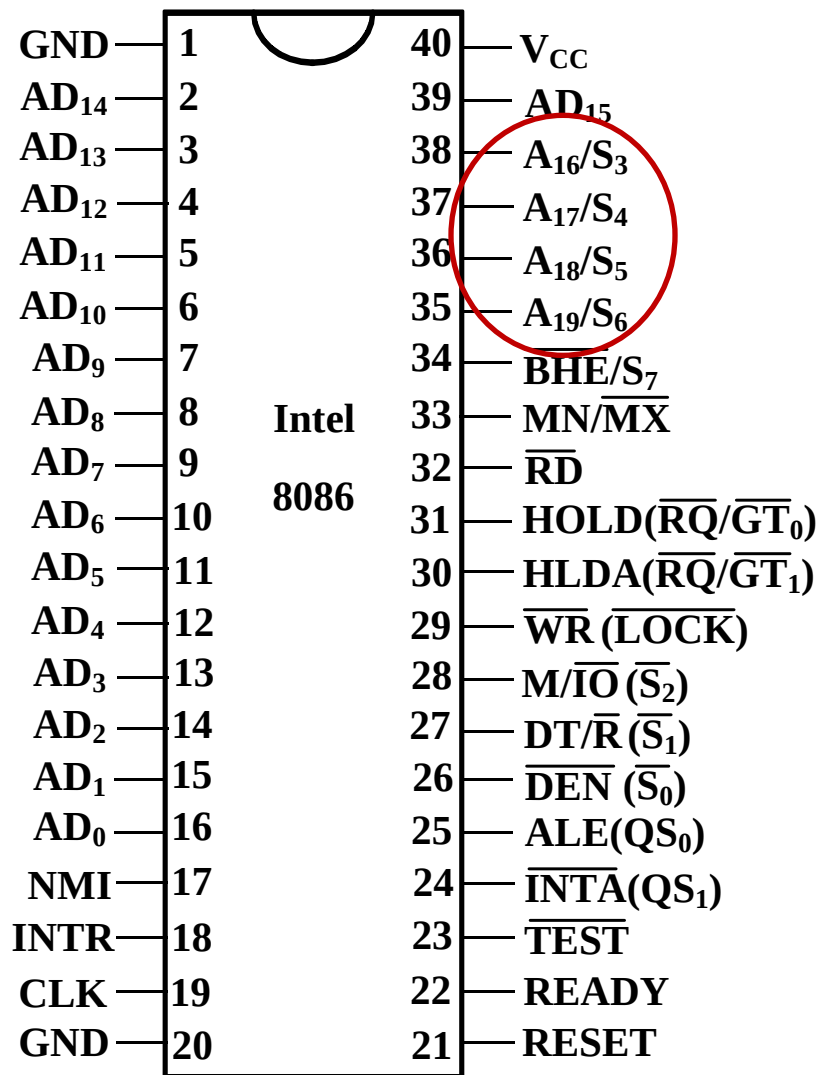
- 低16位地址/数据的复用引脚线
- 三态、双向
- 采用分时复用法来实现对地址线 and 数据线的复用



1、两种工作模式的公共引脚

A₁₉/S₆ ~ A₁₆/S₃

- 高4位地址/状态的复用引脚线
- 三态、输出
- 采用分时复用方法来实现对地址线和状态线的复用



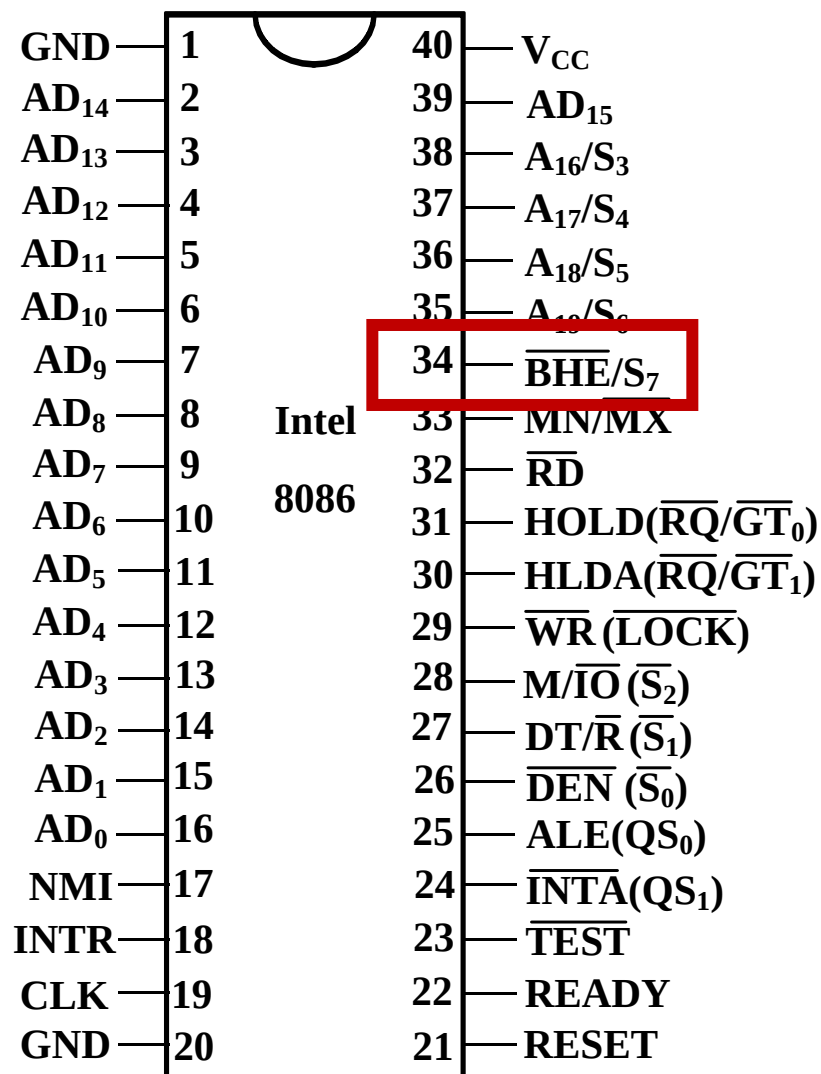
- 状态信息**S₆**总是为低电平。**S₅**反映当前允许中断标志的状态，允许中断时**S₅=1**，禁止中断时**S₅=0**。**S₄**与**S₃**一起指示当前哪一个段寄存器被使用。其规定如表所示。

S ₄	S ₃	当前正在使用的段寄存器名
0	0	ES
0	1	SS
1	0	CS或未用
1	1	DS

1、两种工作模式的公共引脚

$\overline{\text{BHE}}/\text{S}_7$

- 高8位数据总线允许/状态的复用引脚线
- 三态、输出
- $\overline{\text{BHE}}$ 与 A_0 结合使用



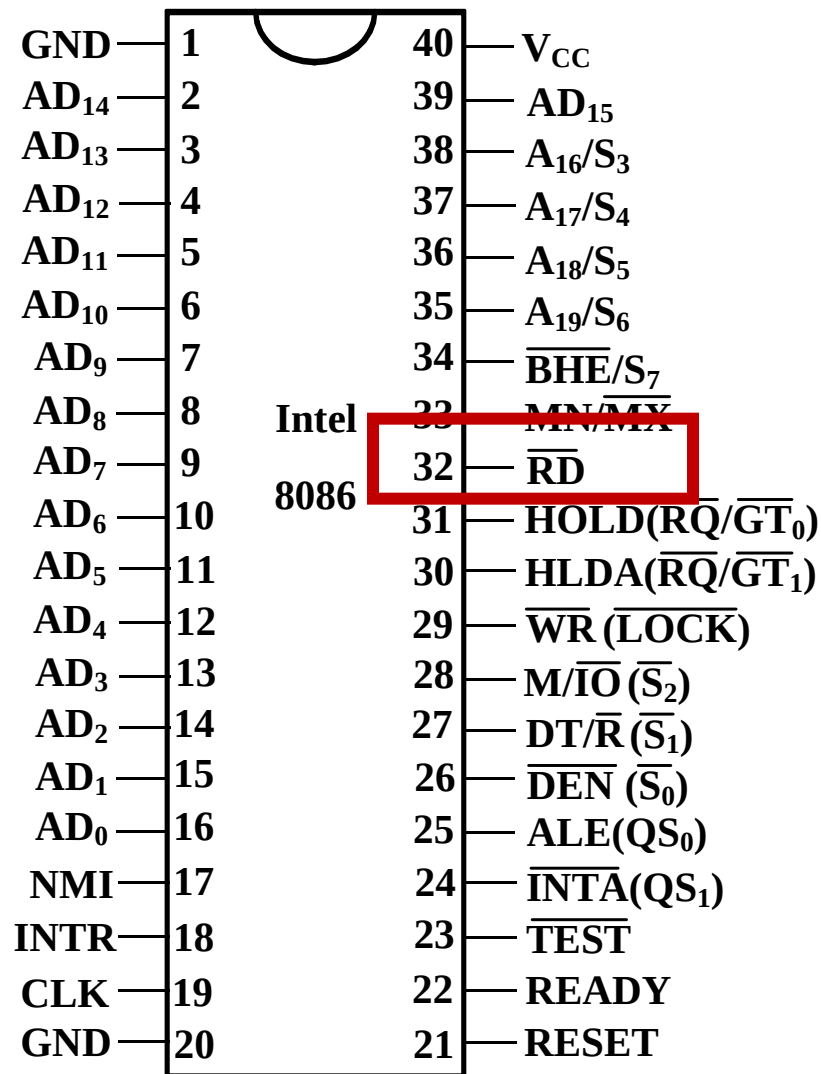
$\overline{\text{BHE}}$ 与 A_0 结合使用

$\overline{\text{BHE}}$	AD_0	有效的数据引脚	操作
0	0	$\text{AD}_{15} \sim \text{AD}_0$	从偶地址读/写 一个字
1	0	$\text{AD}_7 \sim \text{AD}_0$	从偶地址读/写 一个字节
0	1	$\text{AD}_{15} \sim \text{AD}_8$	从奇地址读/写 一个字节
0	1	$\text{AD}_{15} \sim \text{AD}_8$ (第一个总线周期读 到该字数据的低8位)	从奇地址读/写 一个字
1	0	$\text{AD}_7 \sim \text{AD}_0$ (第二个总线周期读 到该字数据的高8位)	

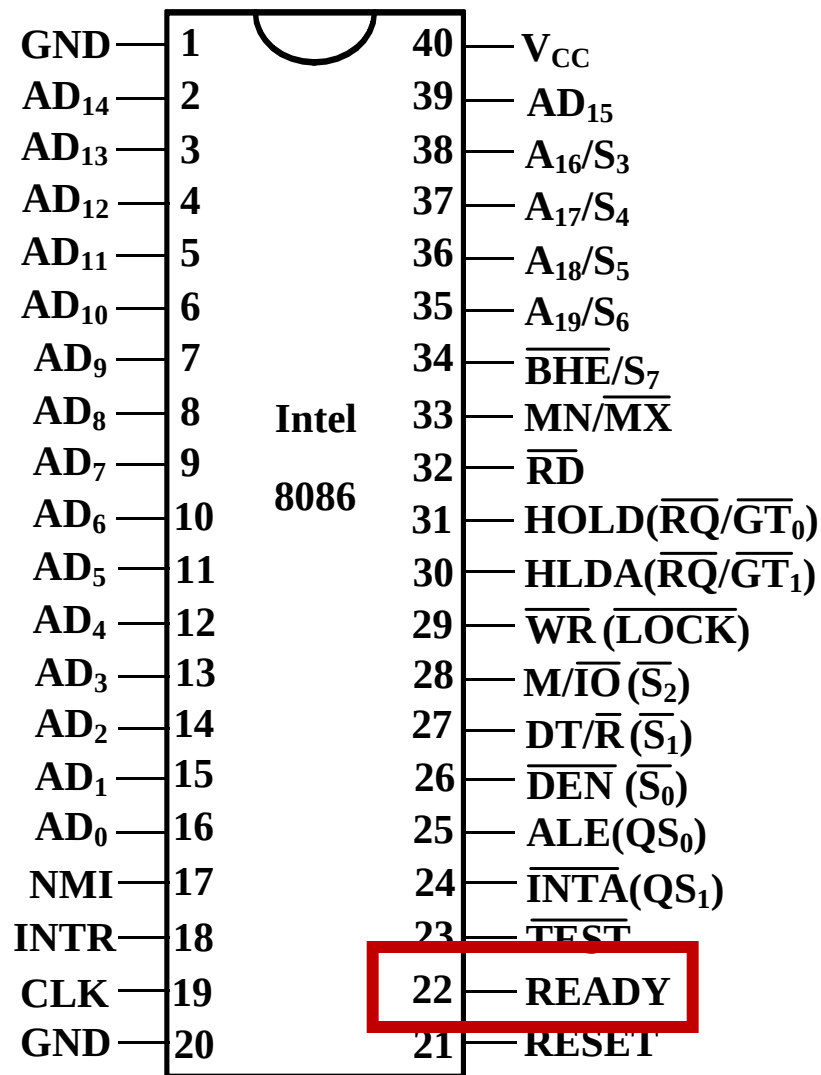
1. 两种工作模式的公共引脚

$\overline{\text{RD}}$

- 读信号
- 三态、输出
- 低电平时有效
- 有效时表示对存储器或I/O进行读操作



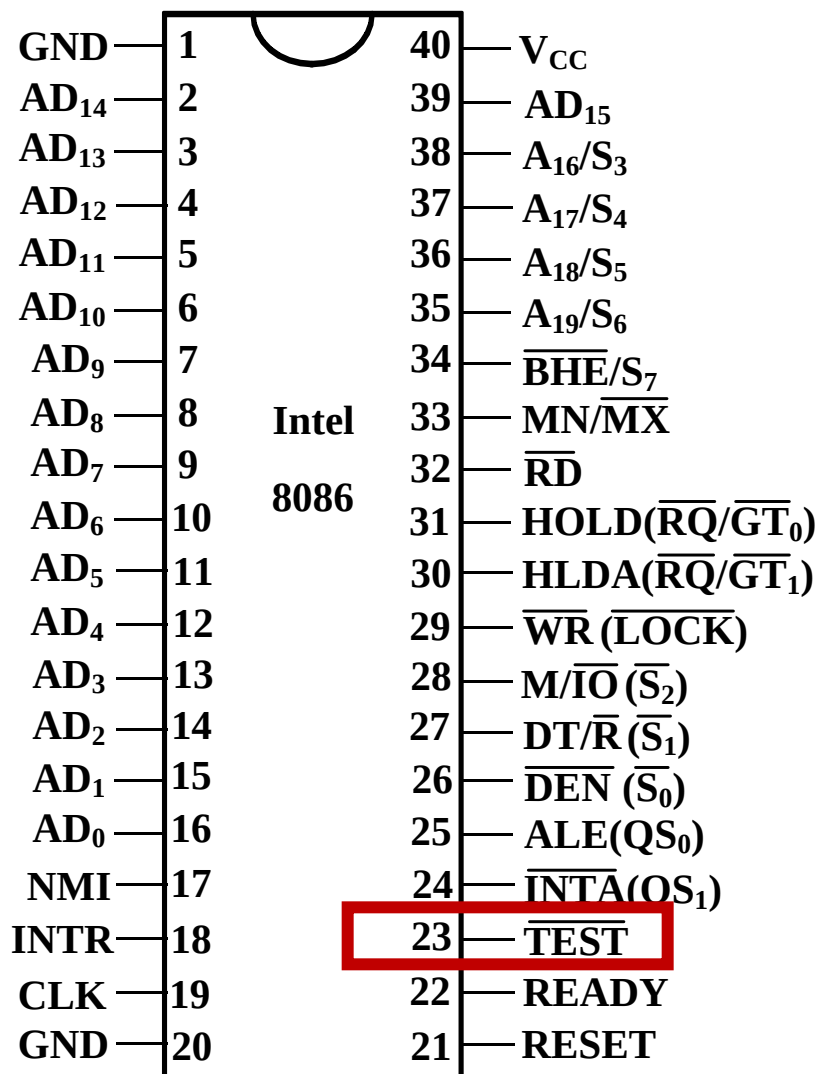
1. 两种工作模式的公共引脚



READY

- 准备就绪信号
- 输入
- 高电平时有效
- 有效时表示存储器或I/O设备准备就绪

1. 两种工作模式的公共引脚



TEST

- 测试信号
- 输入
- 低电平时有效
- 与WAIT指令配合使用

1. 两种工作模式的公共引脚

GND	1	Intel 8086	40	V_{CC}
AD_{14}	2		39	AD_{15}
AD_{13}	3		38	A_{16}/S_3
AD_{12}	4		37	A_{17}/S_4
AD_{11}	5		36	A_{18}/S_5
AD_{10}	6		35	A_{19}/S_6
AD_9	7		34	$\overline{BHE}/S_7$
AD_8	8		33	$MN/\overline{MX}$
AD_7	9		32	$\overline{RD}$
AD_6	10		31	$HOLD(\overline{RQ}/\overline{GT_0})$
AD_5	11		30	$HLDA(\overline{RQ}/\overline{GT_1})$
AD_4	12		29	$\overline{WR}(\overline{LOCK})$
AD_3	13		28	$M/\overline{IO}(\overline{S_2})$
AD_2	14		27	$DT/\overline{R}(\overline{S_1})$
AD_1	15		26	$\overline{DEN}(\overline{S_0})$
AD_0	16		25	$ALE(QS_0)$
NMI	17		24	$\overline{INTA}(QS_1)$
INTR	18		23	$\overline{TEST}$
CLK	19		22	READY
GND	20		21	RESET

INTR

- 可屏蔽中断请求信号
- 输入
- 高电平时有效
- 标志寄存器中IF位：
 - IF=1, 允许中断
 - IF=0, 禁止中断

1. 两种工作模式的公共引脚

GND	1	Intel 8086	40	V_{CC}
AD ₁₄	2		39	AD ₁₅
AD ₁₃	3		38	A ₁₆ /S ₃
AD ₁₂	4		37	A ₁₇ /S ₄
AD ₁₁	5		36	A ₁₈ /S ₅
AD ₁₀	6		35	A ₁₉ /S ₆
AD ₉	7		34	$\overline{BHE}/S_7$
AD ₈	8		33	MN/ $\overline{MX}$
AD ₇	9		32	$\overline{RD}$
AD ₆	10		31	HOLD($\overline{RQ}/\overline{GT_0}$)
AD ₅	11		30	HLDA($\overline{RQ}/\overline{GT_1}$)
AD ₄	12		29	$\overline{WR}$ ($\overline{LOCK}$)
AD ₃	13		28	M/ $\overline{IO}$ ($\overline{S_2}$)
AD ₂	14		27	DT/ $\overline{R}$ ($\overline{S_1}$)
AD ₁	15		26	$\overline{DEN}$ ($\overline{S_0}$)
AD ₀	16		25	ALE(QS ₀)
NMI	17		24	$\overline{INTA}$ (QS ₁)
INTR	18		23	$\overline{TEST}$
CLK	19		22	READY
GND	20		21	RESET

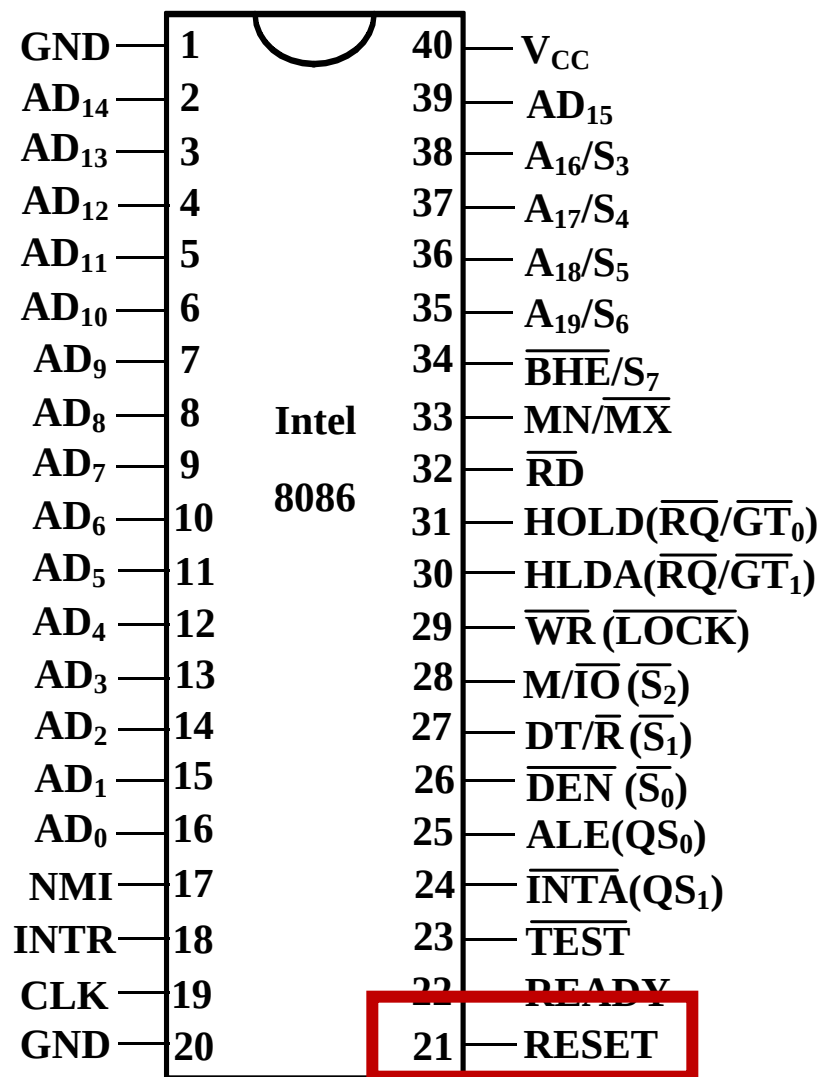
NMI

- 非屏蔽中断请求信号
- 输入
- 上升沿有效
- 不受IF位影响

8086只有NMI和INTR

可以引入外部中断。

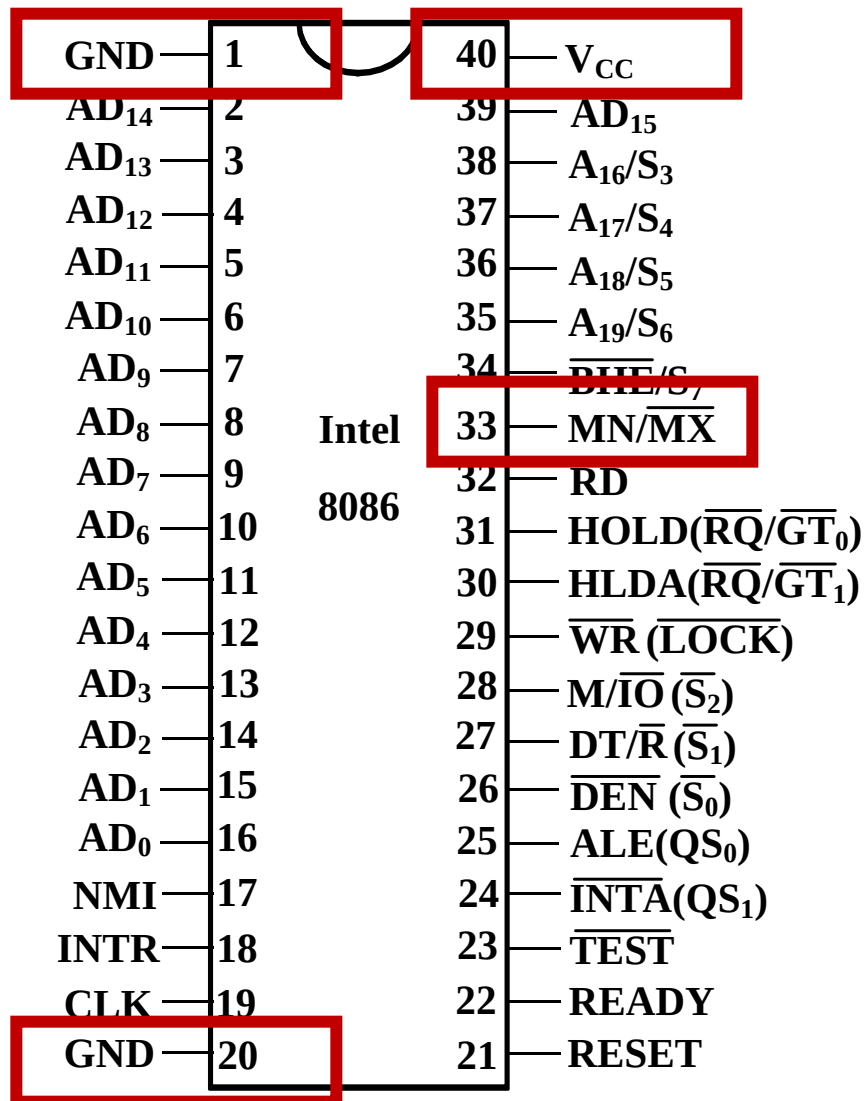
1. 两种工作模式的公共引脚



RESET

- 复位信号
- 输入
- 高电平有效
- 复位后：
 - 指令队列空
 - CS为FFFFH
 - 其它寄存器清零

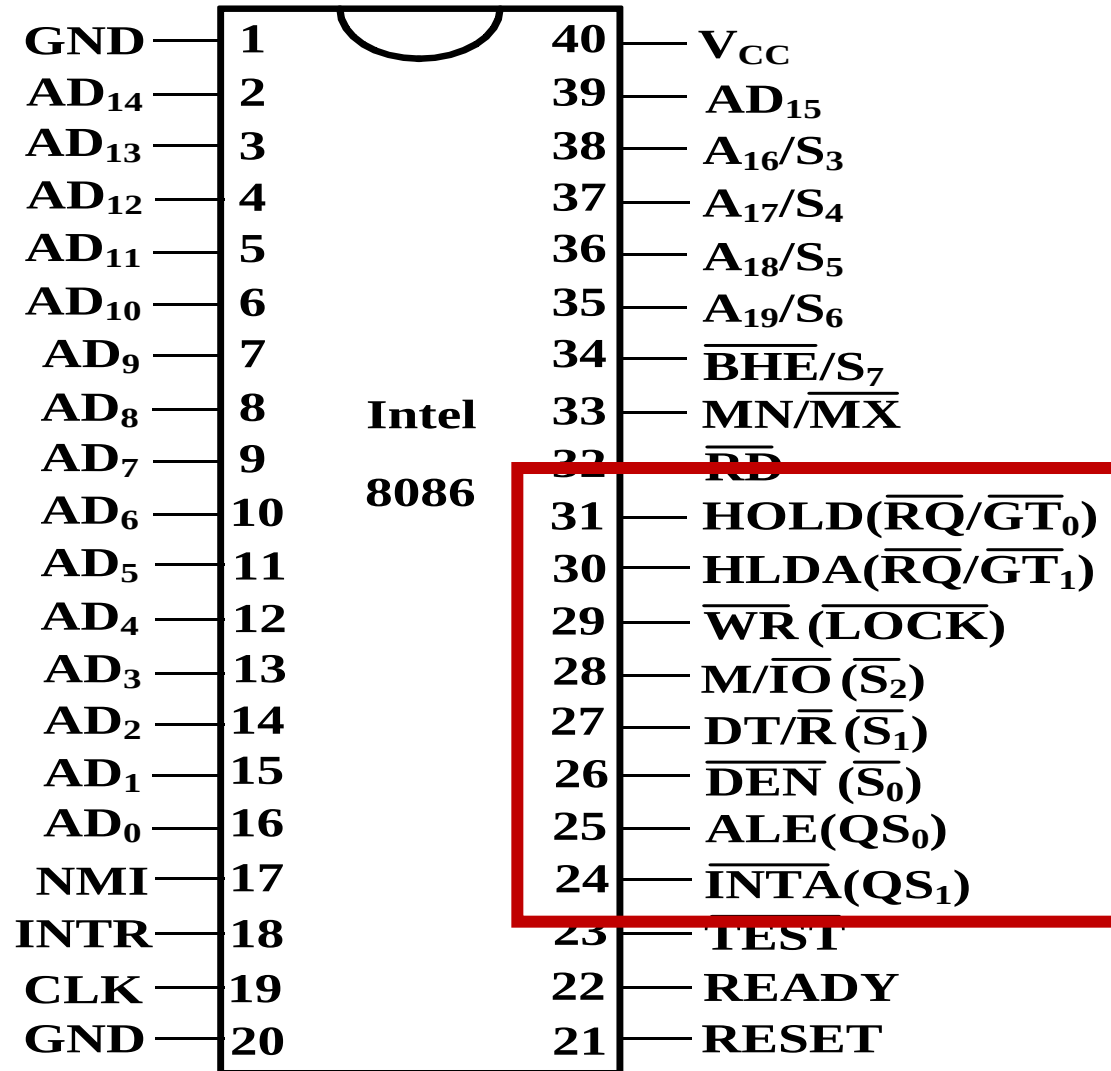
1. 两种工作模式的公共引脚



其它

- CLK输入时钟信号
- V_{CC}输入电源
- GND接地
- MN/ $\overline{MX}$ 最小/最大模式信号，输入。

与模式有关的引脚----- 共有8条



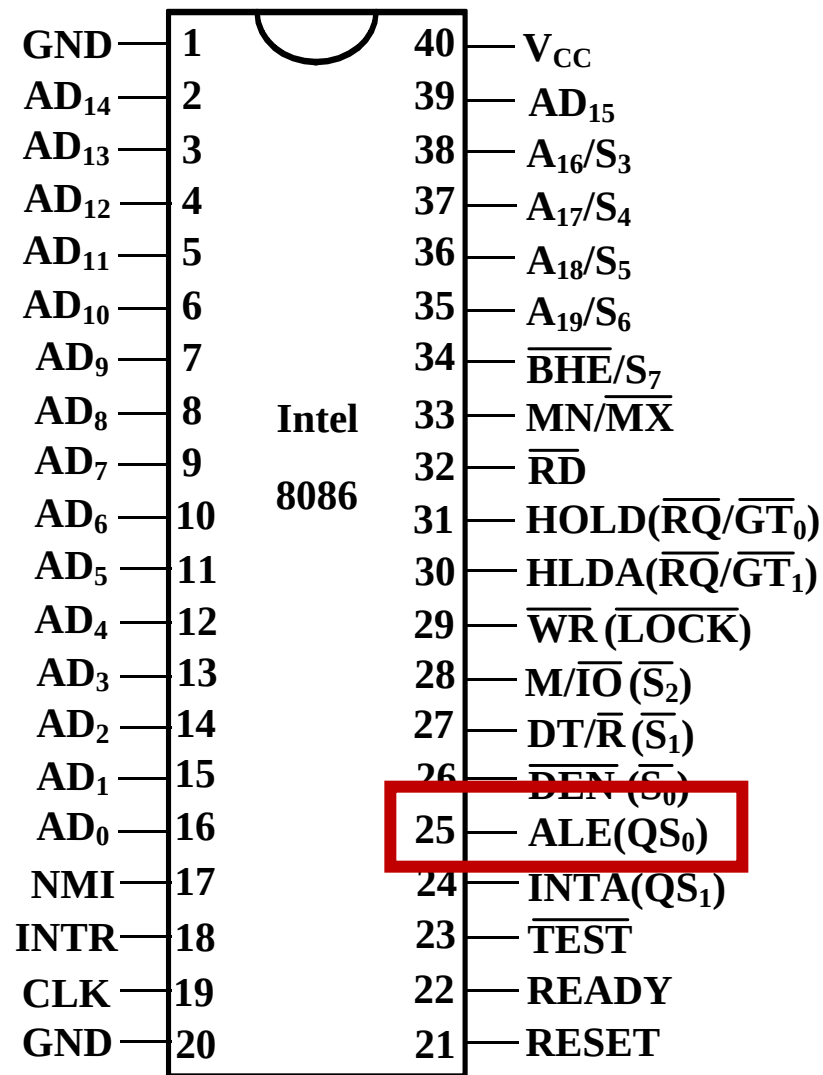
2. 最小模式下的引脚

GND	1	Intel 8086	40	V_{CC}
AD ₁₄	2		39	AD ₁₅
AD ₁₃	3		38	A ₁₆ /S ₃
AD ₁₂	4		37	A ₁₇ /S ₄
AD ₁₁	5		36	A ₁₈ /S ₅
AD ₁₀	6		35	A ₁₉ /S ₆
AD ₉	7		34	$\overline{BHE}/S_7$
AD ₈	8		33	$\overline{MN}/\overline{MX}$
AD ₇	9		32	$\overline{RD}$
AD ₆	10		31	HOLD($\overline{RQ}/\overline{GT_0}$)
AD ₅	11		30	HLDA($\overline{RQ}/\overline{GT_1}$)
AD ₄	12		29	$\overline{WR}$ (LOCK)
AD ₃	13		28	M/ $\overline{IO}$ ($\overline{S_2}$)
AD ₂	14		27	DT/ $\overline{R}$ ($\overline{S_1}$)
AD ₁	15		26	$\overline{DEN}$ ($\overline{S_0}$)
AD ₀	16		25	ALE(OS ₀)
NMI	17		24	$\overline{INTA}$ (QS ₁)
INTR	18		23	$\overline{TEST}$
CLK	19		22	READY
GND	20		21	RESET

$\overline{INTA}$

- 中断响应信号
- 输出
- 低电平有效

2. 最小模式下的引脚



ALE

- 地址锁存信号
- 输出
- 下降沿时锁存地址

2、最小模式下的引脚

GND	1	Intel 8086	40	V_{CC}
AD ₁₄	2		39	AD ₁₅
AD ₁₃	3		38	A ₁₆ /S ₃
AD ₁₂	4		37	A ₁₇ /S ₄
AD ₁₁	5		36	A ₁₈ /S ₅
AD ₁₀	6		35	A ₁₉ /S ₆
AD ₉	7		34	$\overline{BHE}/S_7$
AD ₈	8		33	MN/ $\overline{MX}$
AD ₇	9		32	$\overline{RD}$
AD ₆	10		31	HOLD($\overline{RQ}/\overline{GT_0}$)
AD ₅	11		30	HLDA($\overline{RQ}/\overline{GT_1}$)
AD ₄	12		29	$\overline{WR}$ (LOCK)
AD ₃	13		28	M/ $\overline{IO}$ ($\overline{S_2}$)
AD ₂	14		27	DT/ $\overline{R}$ ($\overline{S_1}$)
AD ₁	15		26	$\overline{DEN}$ ($\overline{S_0}$)
AD ₀	16		25	ALE(QS ₀)
NMI	17		24	$\overline{INTA}$ (QS ₁)
INTR	18		23	$\overline{TEST}$
CLK	19		22	READY
GND	20		21	RESET

$\overline{DEN}$

- 数据允许信号
- 输出，三态
- 低电平有效

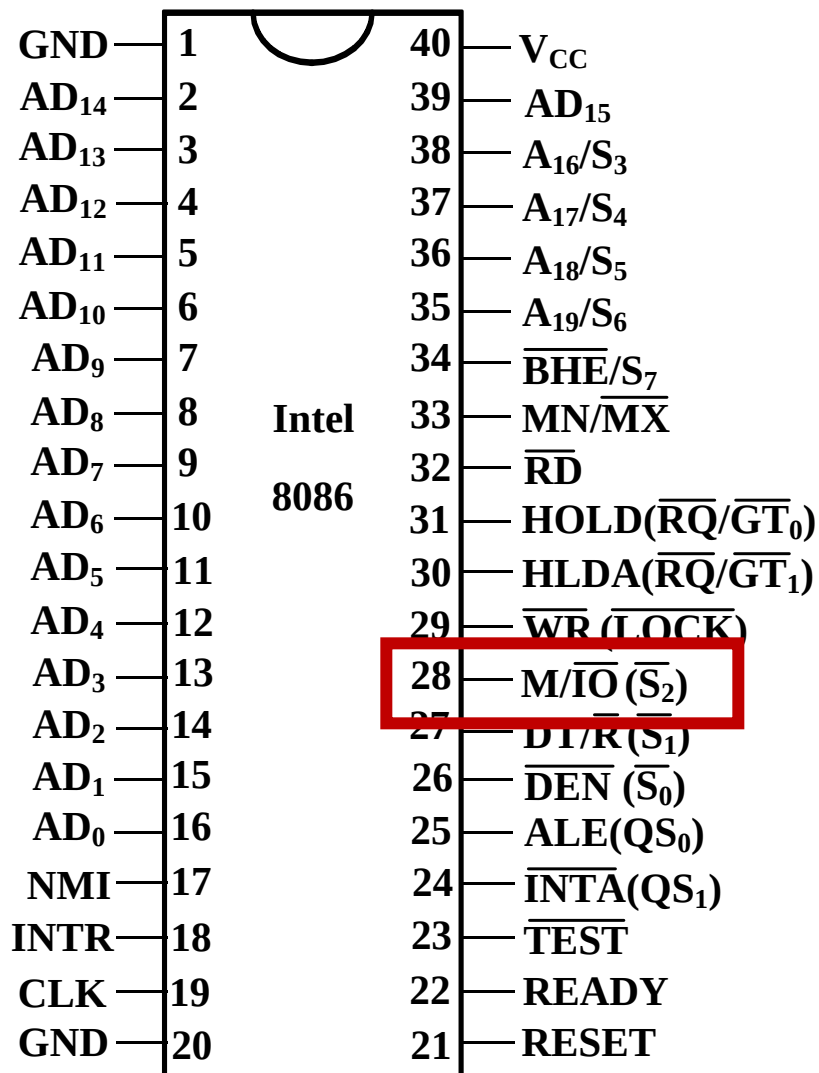
2. 最小模式下的引脚

GND	1	Intel 8086	40	V_{CC}
AD ₁₄	2		39	AD ₁₅
AD ₁₃	3		38	A ₁₆ /S ₃
AD ₁₂	4		37	A ₁₇ /S ₄
AD ₁₁	5		36	A ₁₈ /S ₅
AD ₁₀	6		35	A ₁₉ /S ₆
AD ₉	7		34	$\overline{BHE}/S_7$
AD ₈	8		33	MN/ $\overline{MX}$
AD ₇	9		32	$\overline{RD}$
AD ₆	10		31	HOLD($\overline{RQ}/\overline{GT_0}$)
AD ₅	11		30	HLDA($\overline{RQ}/\overline{GT_1}$)
AD ₄	12		29	$\overline{WR}$ (LOCK)
AD ₃	13		28	M/ $\overline{IO}$ ($\overline{S_2}$)
AD ₂	14		27	DT/ $\overline{R}$ ($\overline{S_1}$)
AD ₁	15		26	DEN (S_0)
AD ₀	16		25	ALE(QS ₀)
NMI	17		24	$\overline{INTA}$ (QS ₁)
INTR	18		23	$\overline{TEST}$
CLK	19		22	READY
GND	20		21	RESET

DT/ $\overline{R}$

- 数据发送/接收控制信号
- 输出，三态
- 有效时控制数据传送方向

2. 最小模式下的引脚



M/ $\overline{\text{IO}}$

➤ 访问M或I/O

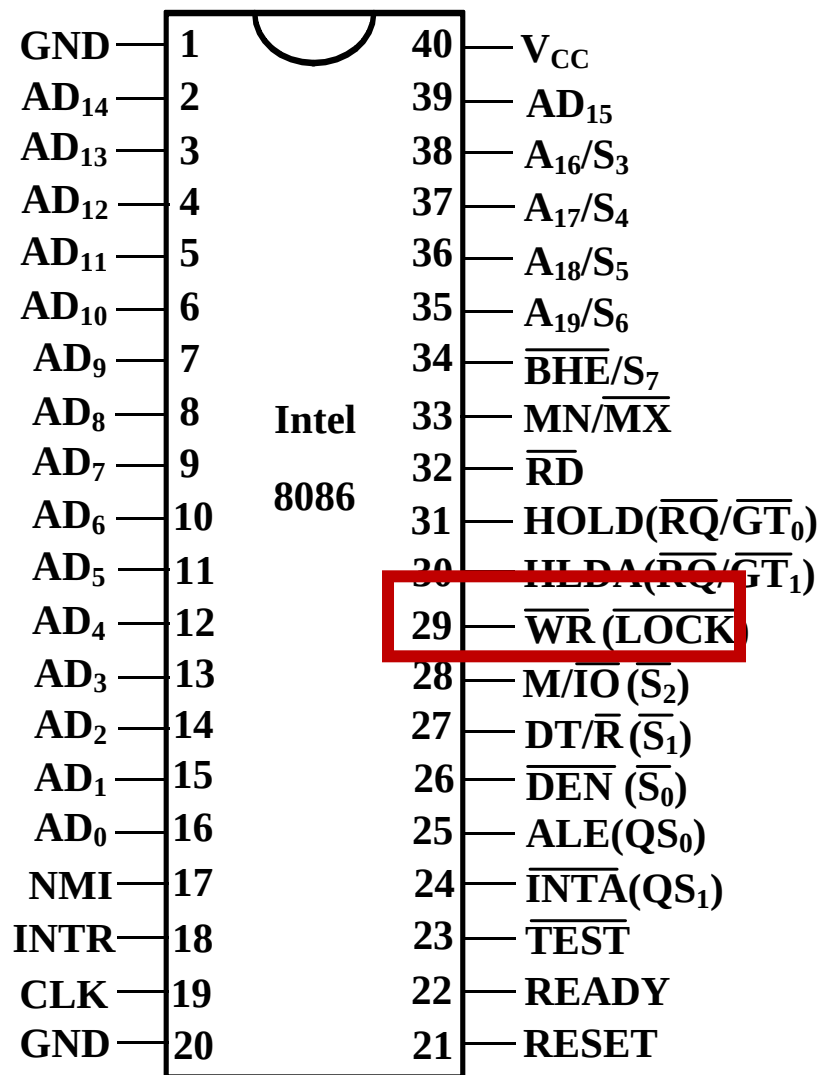
控制信号

➤ 输出，三态

➤ =1，访问M

➤ =0，访问I/O

2. 最小模式下的引脚



➤ 写信号

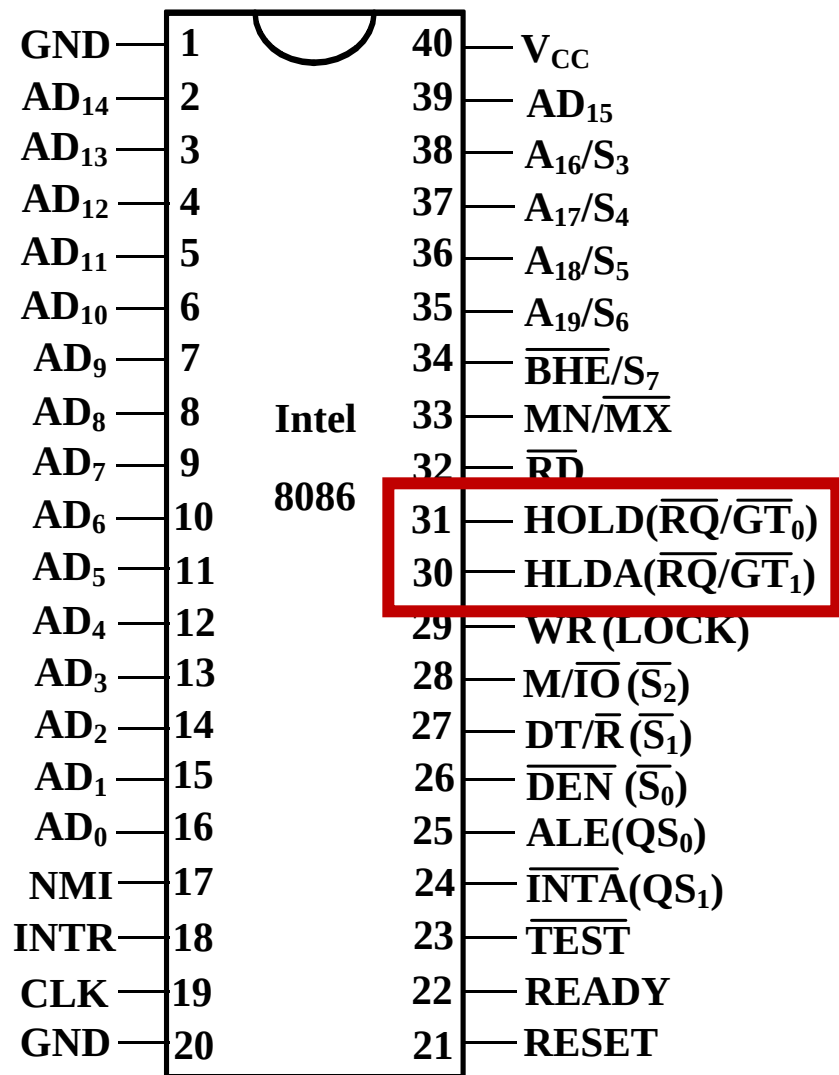
➤ 输出，三态

➤ 低电平有效

➤ 有效时表示

CPU正在写

2. 最小模式下的引脚



HOLD和HLDA

➤ HOLD输入，总线保持请求信号

➤ HLDA输出，总线响应信号

3. 最大模式下的引脚

GND	1		40	V_{CC}
AD_{14}	2		39	AD_{15}
AD_{13}	3		38	A_{16}/S_3
AD_{12}	4		37	A_{17}/S_4
AD_{11}	5		36	A_{18}/S_5
AD_{10}	6		35	A_{19}/S_6
AD_9	7		34	$\overline{BHE}/S_7$
AD_8	8	Intel	33	$MN/\overline{MX}$
AD_7	9	8086	32	$\overline{RD}$
AD_6	10		31	$HOLD(\overline{RQ}/\overline{GT_0})$
AD_5	11		30	$HLDA(\overline{RQ}/\overline{GT_1})$
AD_4	12		29	$\overline{WR}(\overline{LOCK})$
AD_3	13		28	$M/\overline{IO}(\overline{S_2})$
AD_2	14		27	$DT/\overline{R}(\overline{S_1})$
AD_1	15		26	$\overline{DEN}(\overline{S_0})$
AD_0	16		25	$ALE(QS_0)$
NMI	17		24	$\overline{INTA}(QS_1)$
INTR	18		23	TEST
CLK	19		22	READY
GND	20		21	RESET

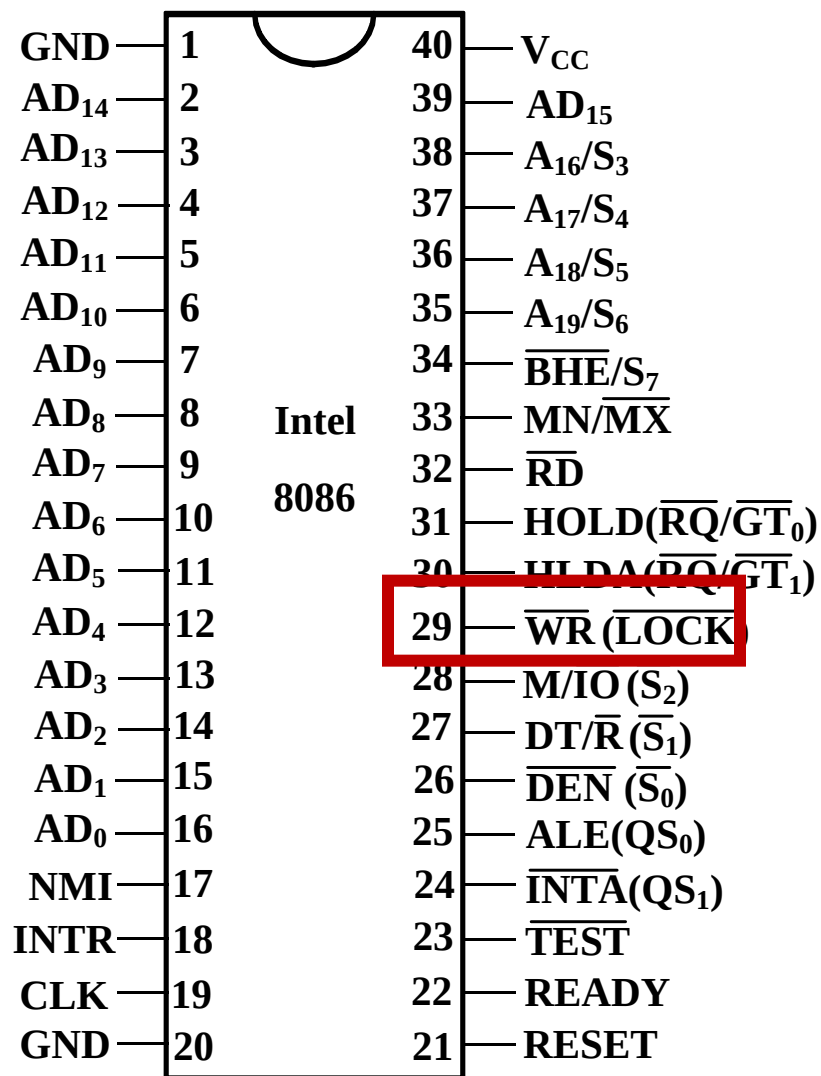
QS_1, QS_0

- 指令队列状态信号
- 输出
- 组合见表

QS₁和QS₀的组合与对应的操作

QS ₁	QS ₀	操作
0	0	无操作
0	1	从指令队列的第一个字节中取走代码
1	0	队列为空
1	1	除第一个字节外，还取走了后续字节中的代码

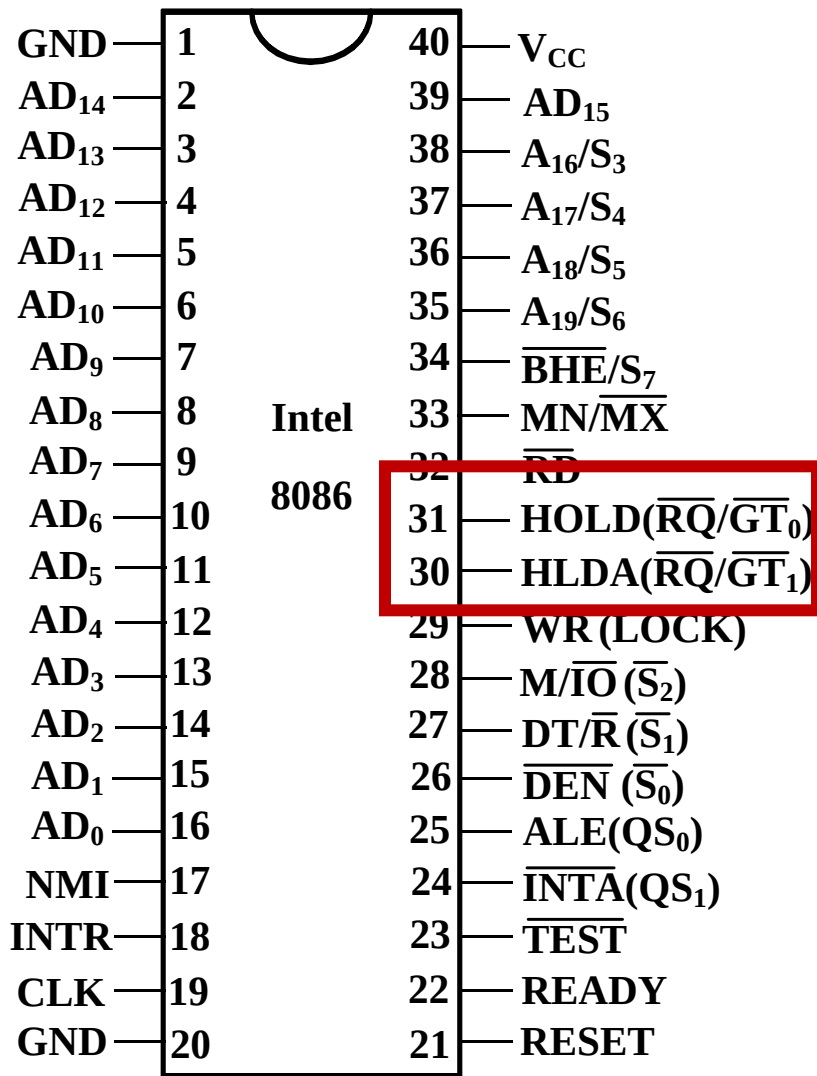
3. 最大模式下的引脚



LOCK

- 总线封锁信号
- 输出，三态
- 低电平有效

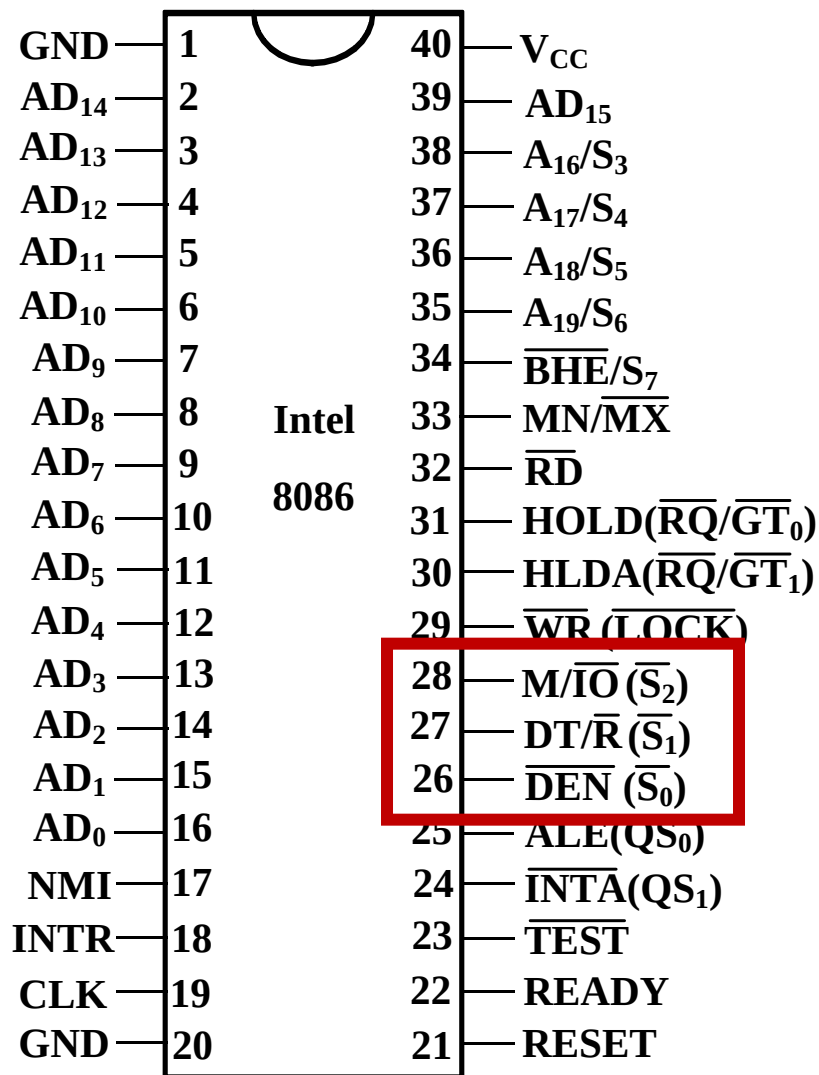
3. 最大模式下的引脚



$\overline{\text{RQ}}/\text{GT}$

- 总线请求/允许信号
- 双向
 - 输入：请求
 - 输出：响应
- 共有2个同样的引脚
- $\text{RQ}/\overline{\text{GT}}_0$ 优先级高

3. 最大模式下的引脚



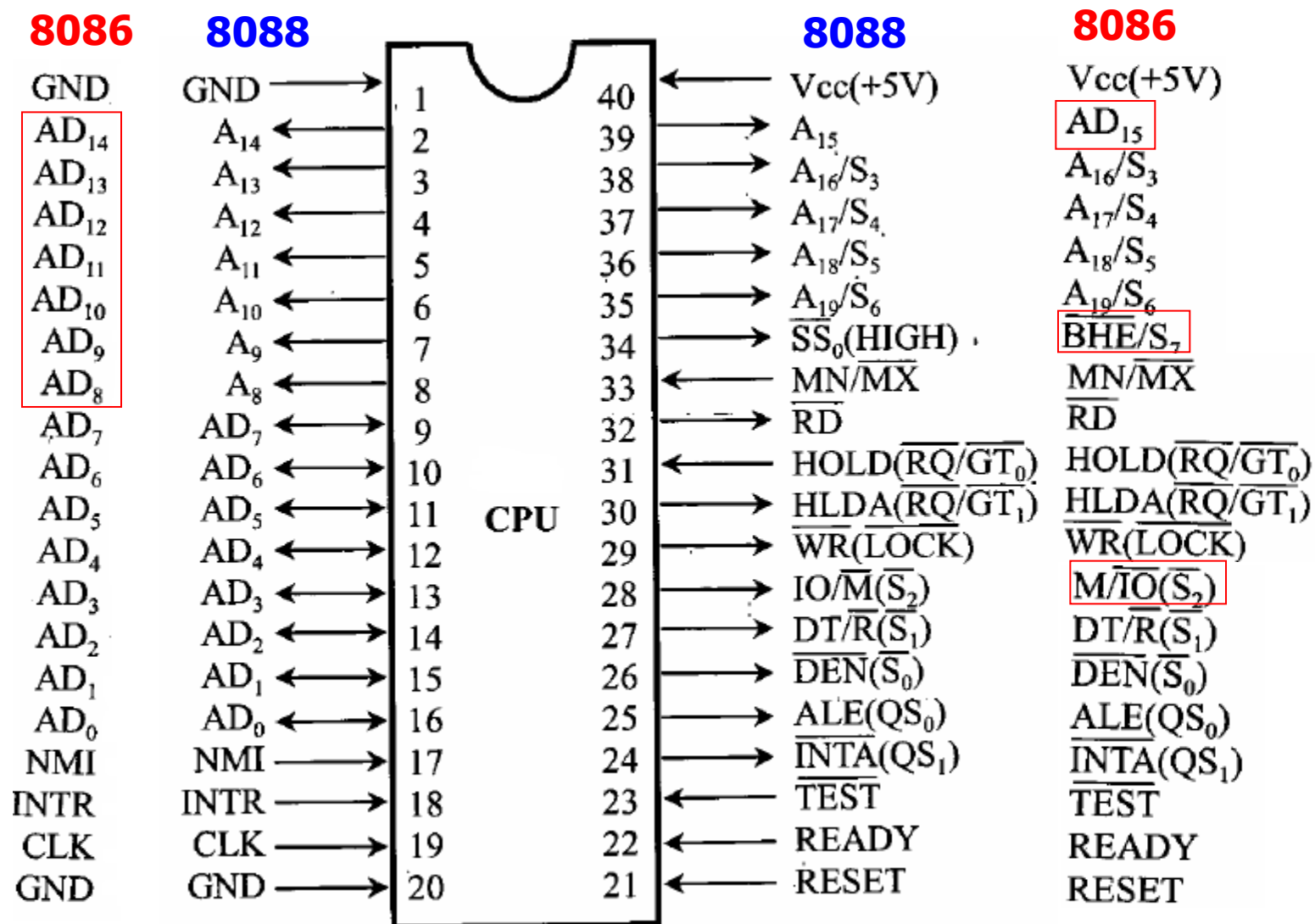
$\overline{S_{2\sim0}}$

- 总线周期状态信号
- 输出信号
- 组合见表

$\overline{S}_2$ 、 $\overline{S}_1$ 、 $\overline{S}_0$ 的组合与对应的操作

$\overline{S}_2$	$\overline{S}_1$	$\overline{S}_0$	操作
0	0	0	中断响应
0	0	1	读I/O端口
0	1	0	写I/O端口
0	1	1	暂停
1	0	0	取指
1	0	1	读存储器
1	1	0	写存储器
1	1	1	无作用

8088与8086的引脚差别



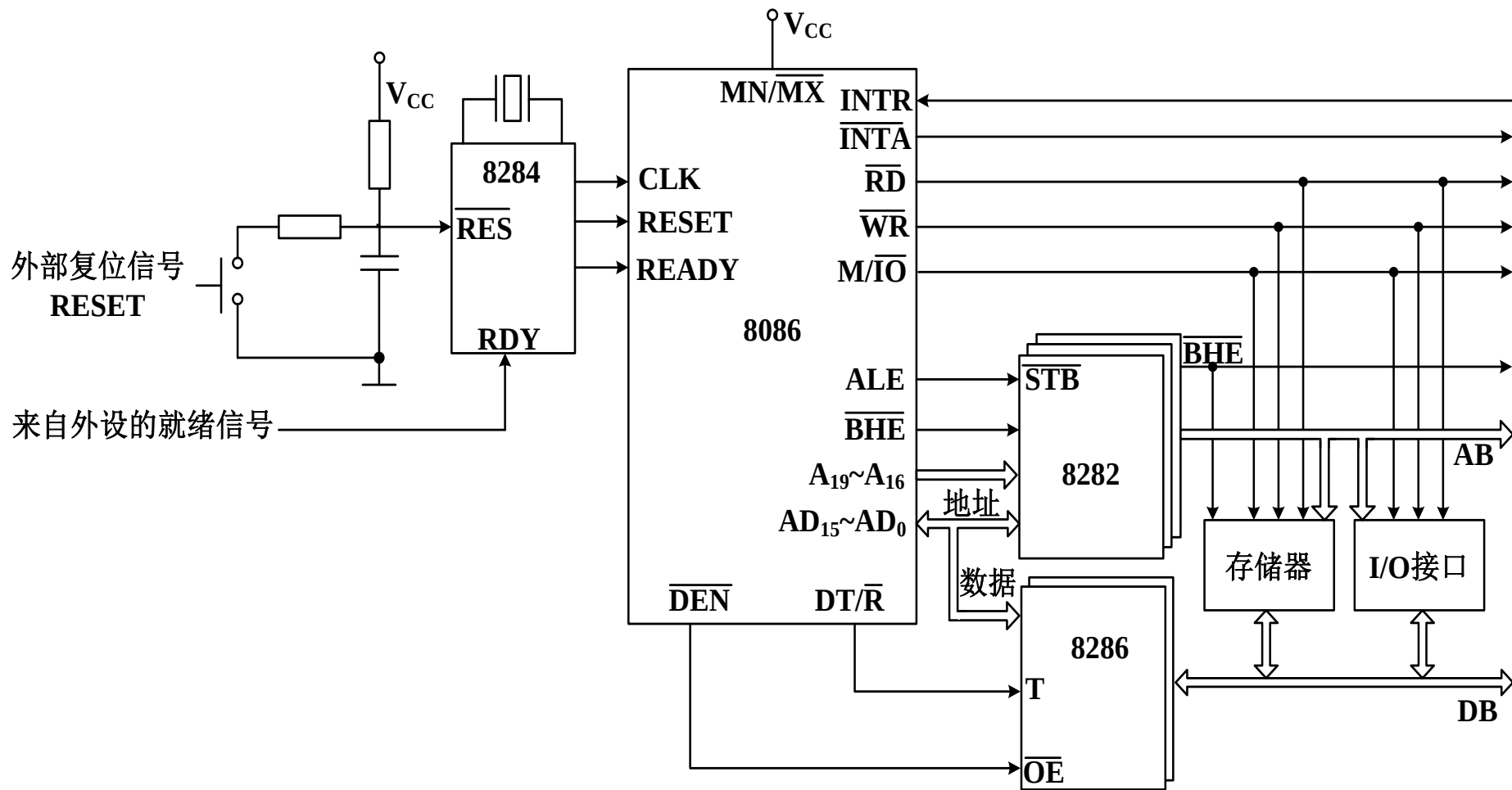
2.3 8086微型计算机系统

2.3.1 8086微型计算机系统的硬件组成

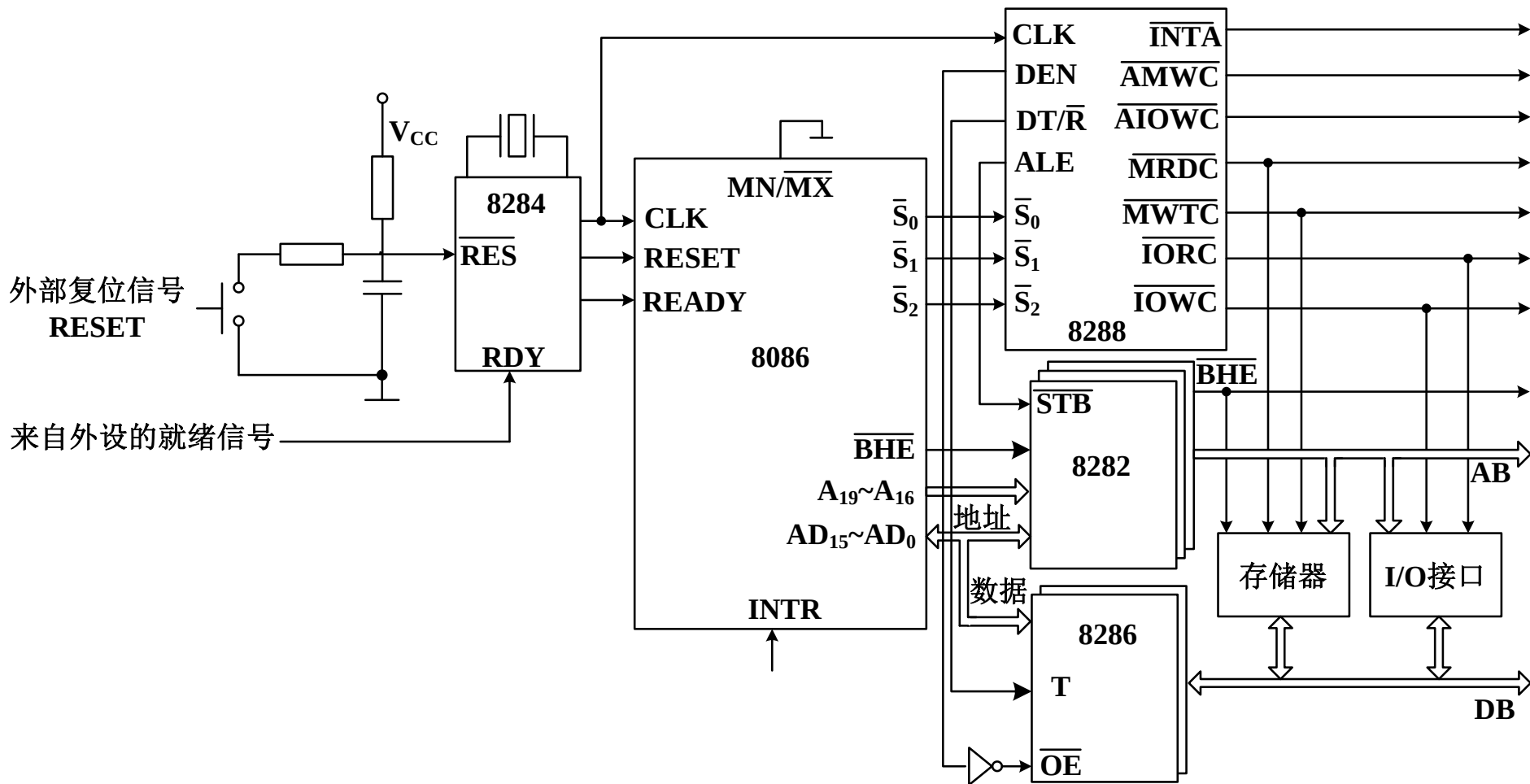
1. 系统硬件组成的特点

- **8086**微型计算机系统的硬件组成出了包括**8086**微处理器外，还需要其他的部件。
- **8086**不同的工作模式对系统的硬件组成有不同的要求，其中共同之处有：
 - 时钟发生器**8284** (1片，提供时钟)
 - 地址锁存器**8282** (3片，锁存地址信息)
 - 数据收发器**8286** (2片，增加数据总线的驱动能力)

2. 最小模式系统的硬件组成



3. 最大模式系统的硬件组成



2.3.2 8086微型计算机系统的存储器组织

- 存储器是计算机的记忆部件，用来存放程序和数据。
- 按所在的位置，存储器可以分成主存储器和辅助存储器。
 - 主存储器存放当前正在执行的程序和使用的数据，**CPU**可以直接存取，它由半导体存储器芯片构成，其成本高，容量小，但速度快。
 - 辅助存储器可用来长期保存大量程序和数据，**CPU**需要通过**I/O**接口访问，它由磁盘或光盘构成，其成本低，容量大，但速度较慢。

1. 8086的存储器空间

■ **8086**中本身没有存储器，这里是指以**8086**为**CPU**的计算机系统的存储空间；

■ 存储单元

- **8086**的存储器空间由若干存储单元组成；
- 每个**字节**为一个内存单元，它具有唯一的地址码。

■ 存储单元的内容

- 指的是存放在存储单元中的数据；
- 数据可以是字节、字和双字。

■ 存储容量

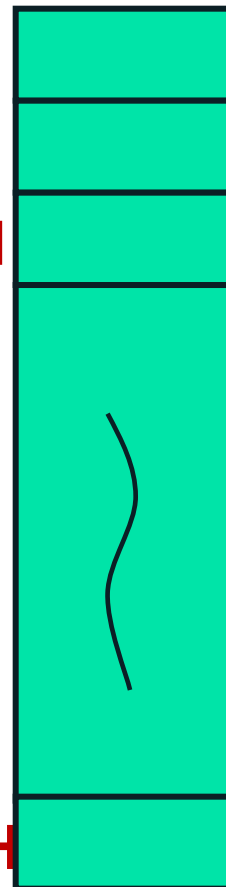
- **存储器能够存放数据的最大字节数；**
- **与地址线多少相关，8086有地址线20根，故8086****FFFFFH**
计算机系统中，可寻址的存储空间最大可以是
 $2^{20}=1\text{M}$ 字节。

00000H

00001H

0000FH

FFFFFH



1. 8086的存储器空间

■ 存储单元地址

- 即为存储单元的编号，用无符号整数来表示（编程用十六进制表示）；
- **8086物理地址范围从00000H~FFFFFFH。**

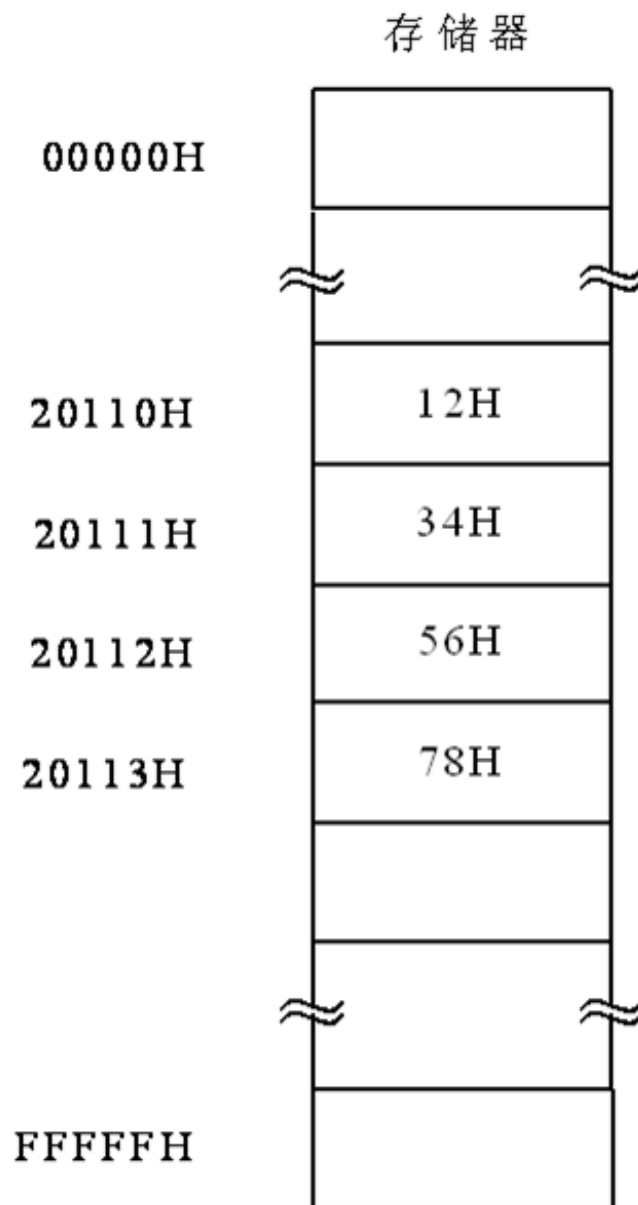
■ 数据在存储单元的存放

- **高高低低**（高字节存放在高地址，低字节存放在低地址）；
- 多字节数据以最低字节的地址为准；
- **8086CPU**对字/字节的读写以从偶地址开始，**16**位为单位进行操作。

1. 8086的存储器空间

如图所示:

- **20110H**单元~**20113H**单元存放的内容依次是**12H**、**34H**、**56H**、**78H**
- **(20110H) = 12H**表示**字节**单元
20110H的内容是**12H**;
- **(20110H) = 3412H**表示**字**单元
20110H的内容是**3412H**;
- **(20110H) = 78563412H**表示**双字**单元
20110H的内容是**78563412H**。
- **(20111H) = 34H**表示**字节**单元
20111H的内容是**34H**;
- **(20111H) = 5634H**表示**字**单元
20111H的内容是**5634H**;

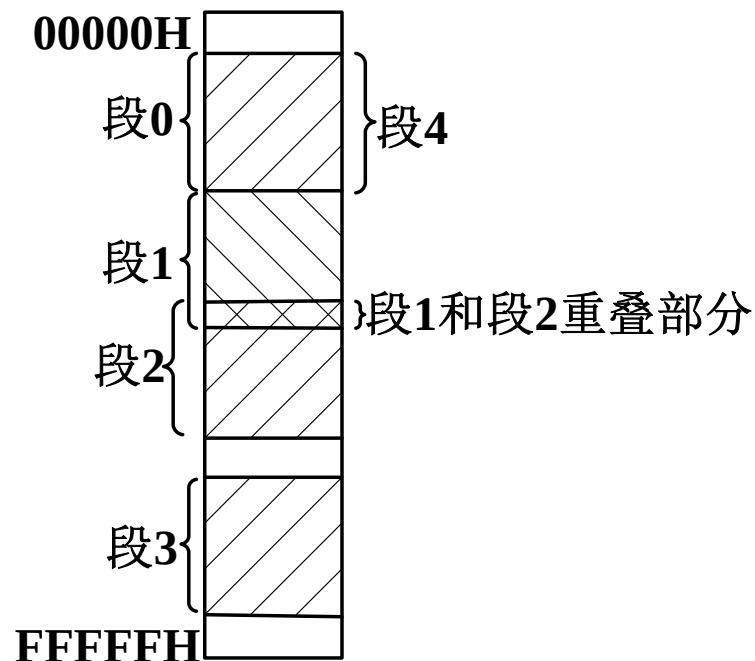


2. 存储器的分段管理

- 尽管**8086CPU**提供**20**位地址，但**8086**中用来存放地址的寄存器如**IP**、**SP**、**BX**、**SI**等都是**16**位的，只能直接寻址**64KB**；
- 为了对**1M**个存储单元进行管理，**8086CPU**采用了典型的存储器分段技术。

2. 存储器的分段管理

- 将整个存储器空间分为许多逻辑段
- 每个逻辑段的容量 $\leq 64\text{KB}$ 字节；
- 各个逻辑段之间可以紧密相连，也可以互相重叠；
- 对存储器的寻址操作采用段地址加段内偏移地址的二级寻址方式；
- 对于任何一个物理地址，可以唯一地被包含在一个逻辑段中，也可包含在多个相互重叠的逻辑段中，只要有段地址和段内偏移地址就可以访问到这个物理地址所对应的存储空间。



2. 存储器的分段管理

■ 分段后，对存储器的寻址操作不再直接用**20**位的物理地址，而是采用段地址 + 段内偏移地址的二级寻址方式。

■ 段地址

- 在**8086**存储空间中，把**16**个字节的存储空间称作一节（**Paragraph**），通常分段时要求各个逻辑段从节的整数边界开始，这样段首地址低**4**位应该是“**0000**”，因此就把段首地址的高**16**位称为“段基址（段地址）”
- 段地址是无符号的**16**位二进制数，存放在段寄存器**DS**、**CS**、**SS**或**ES**中，这些段分别称为数据段、代码段、堆栈段和附加段。

2. 存储器的分段管理

■ 偏移地址

- 把某一存储单元相对于段首地址的偏移量称为偏移地址（也称有效地址**EA**）。
- 偏移地址也是无符号的**16**位二进制数，存放在**IP**、**SP**、**BX**、**SI**、**DI**、**BP**中或直接出现在指令中。

2. 存储器的分段管理

■ 逻辑地址

- 采用分段结构的存储器中，把通过段地址和偏移地址来表示的存储单元的地址称为逻辑地址，记为：

段地址:偏移地址

- 逻辑地址是物理地址的一种表示方式，不是唯一的。
- 例如：

逻辑地址**2000H:1300H**、**2120H:0100H**和

2100H:0300H表示的是同一个存储单元**21300H**。

2. 存储器的分段管理

■ 物理地址

- 物理地址（**PA**）是存储单元的绝对地址，**20**位，是**CPU**访问存储器的**唯一的实际地址**，每个存储单元对应一个物理地址；
- **8086**的存储空间物理地址范围是**00000H~FFFFFFH**

2. 存储器的分段管理

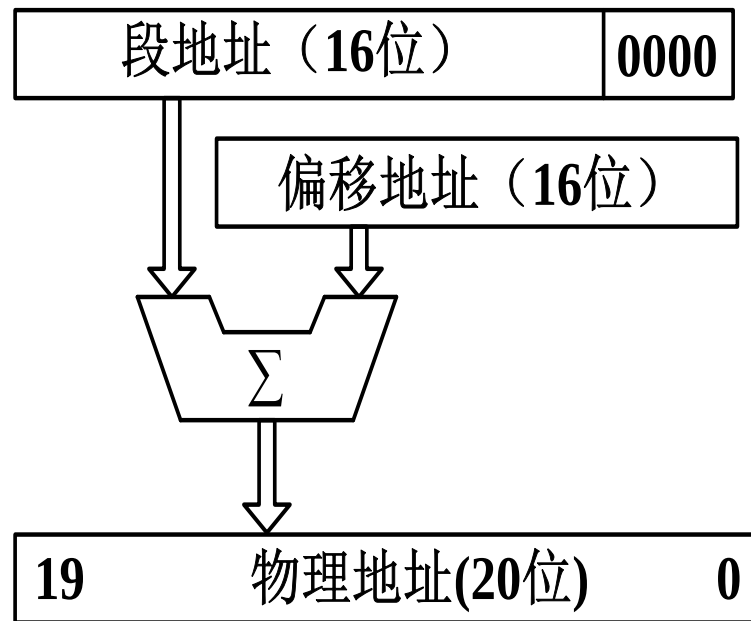
物理地址由逻辑地址形成

■ **物理地址 = 段地址 × 10H + 偏移地址**

■ **CPU**中，这个过程由地址加法器完成；

■ 例如，逻辑地址**2000H:1300H**，所代表的物理地址为：

$$\begin{aligned} \text{PA} &= \mathbf{2000H} \times \mathbf{2^4} + \mathbf{1300H} \\ &= \mathbf{20000H} + \mathbf{1300H} \\ &= \mathbf{21300H} \end{aligned}$$



- 微型计算机系统的外部设备各式各样，**8086CPU**和外部设备之间是通过**I/O**接口芯片进行联系；

- 每个**I/O**芯片上都有一个端口或几个端口，一个端口往往对应于芯片上的一个寄存器或一组寄存器；

- 微机系统要为每个端口分配一个地址，我们把这个地址叫作**端口地址或端口号**；

- 各个端口号具有唯一性；

I/O端口有两种编址方式:

■ 统一编址

- 将**I/O**端口和存储单元统一编址，即把**I/O**端口也看作是存储单元。
- 优点：可利用存储器的寻址方式来寻址**I/O**端口。
- 缺点：**I/O**端口占用了存储空间，而且进行**I/O**操作时，因地址编码较长，导致速度较慢。

■ 独立编址

- 将**I/O**端口和存储单元分开编址，即**I/O**端口空间与存储空间相互独立。
- 优点：不占用内存空间；使用**I/O**指令，程序清晰，很容易看出是**I/O**操作还是存储器操作；译码电路比较简单。
- 缺点：只能用专门的**I/O**指令，访问端口的方法不如访问存储器的方法多。

-
- **8086**采用独立编址方式，使用 $A_{15} \sim A_0$ 这**16**条地址线作端口地址，可访问的**I/O**端口最多可有**64K**个**8**位端口或**32K**个**16**位的端口， $A_{19} \sim A_{16}$ 总是为**0**；
 - 一个**8**位端口相当于一个存储器字节，分配**0000** ~ **FFFFH**中的一个地址。任何两个相邻的**8**位端口可以组合成一个**16**位端口，类似于存储器中的字。
 - **8086**中，有专门的指令来访问**I/O**接口，如输入指令**IN**，输出指令**OUT**。

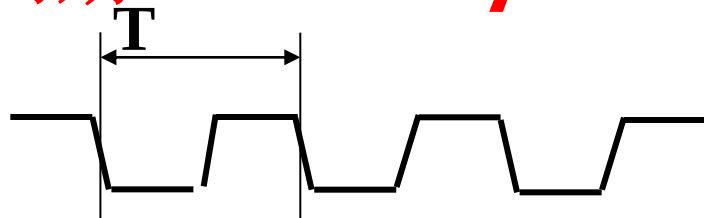
2.4 8086微机系统的总线时序

2.4.1 基本概念

- 微型计算机系统的所有操作都按统一的时钟节拍进行。
- **CPU**执行指令时涉及三种周期：
 - 时钟周期
 - 总线周期
 - 指令周期

1、时钟周期

- 计算机是一个复杂的时序逻辑电路，时序逻辑电路都有“时钟”信号。计算机的“时钟”是由振荡源产生的、幅度和周期不变的节拍脉冲，每个脉冲周期称为**时钟周期（Clock Cycle）**，又称为**T状态**。

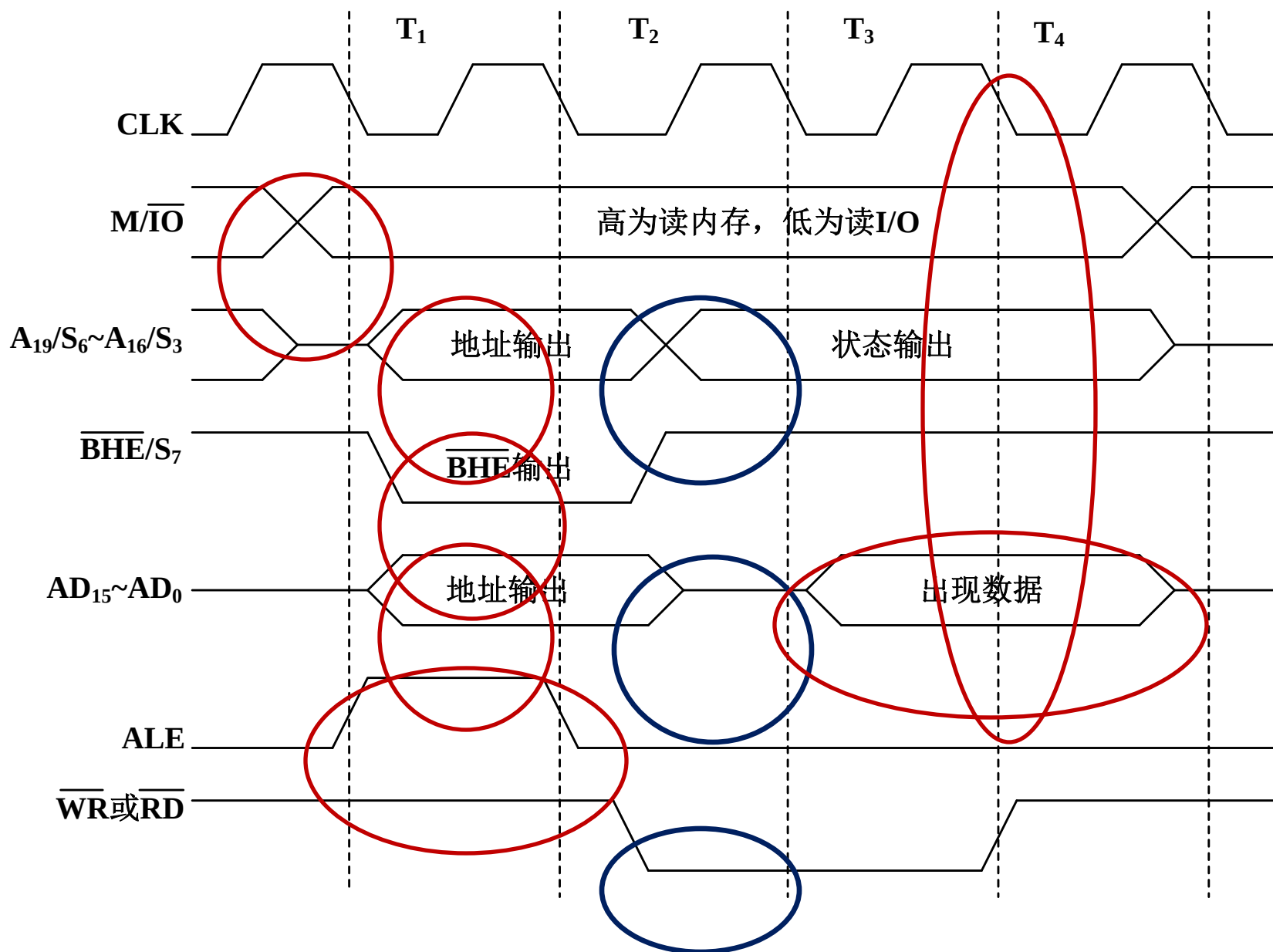


- 计算机是在时钟脉冲的统一控制下，一个节拍一个节拍地工作的。
- 时钟周期是微机系统工作的最小时间单元，取决于系统的主频率，系统完成任何操作所需要的时间，均是时钟周期的整数倍。
- 例如，**IBM-PC**机时钟频率为**4.77MHz**，一个**T状态**为**210ns**

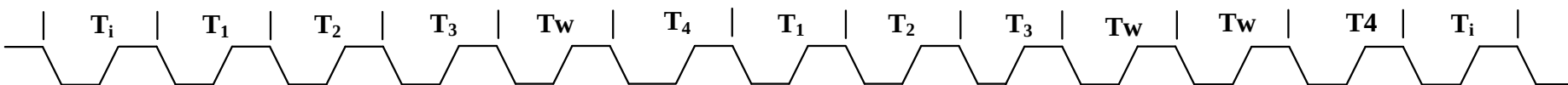
2、总线周期

- 当**CPU**访问**M**或**I/O**端口，需要通过总线进行读或写操作。与**CPU**内部操作相比，通过总线进行的操作需要较长的时间。
- 把**CPU**通过总线进行某种操作的过程称为总线周期（**Bus Cycle**），表示从**M**或**I/O**端口存取一个数据所需的时间。
- 根据总线操作功能的不同，分为：
 - 存储器读周期
 - 存储器写周期
 - **I/O**读周期
 - **I/O**写周期
- 一个基本的总线周期由**4**个时钟周期组成（**T₁**、**T₂**、**T₃**、**T₄**）。

一个基本的总线周期



2、总线周期

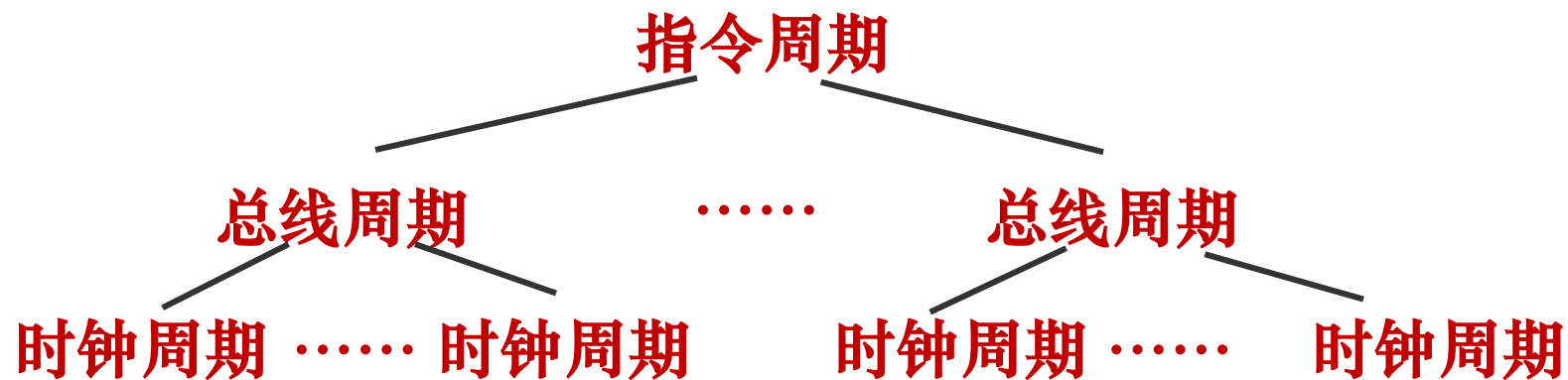


- 在有些情况下，外设或存储器速度较慢，不能及时地配合**CPU**传送数据。这时，外设或存储器会通过“**READY**”信号线在**T3**状态启动之前向**CPU**发一个“数据未准备好”信号，于是**CPU**会在**T3**之后插入**1**个或多个附加的时钟周期**等待状态Tw**。
- 只有在**CPU**和内存或**I/O**接口之间传输数据，以及填充指令队列时，**CPU**才执行总线周期。可见，如果在一个总线周期之后，不立即执行下一个总线周期，那么，系统总线就处在**空闲状态**，此时，执行空闲周期 **T_i** 。

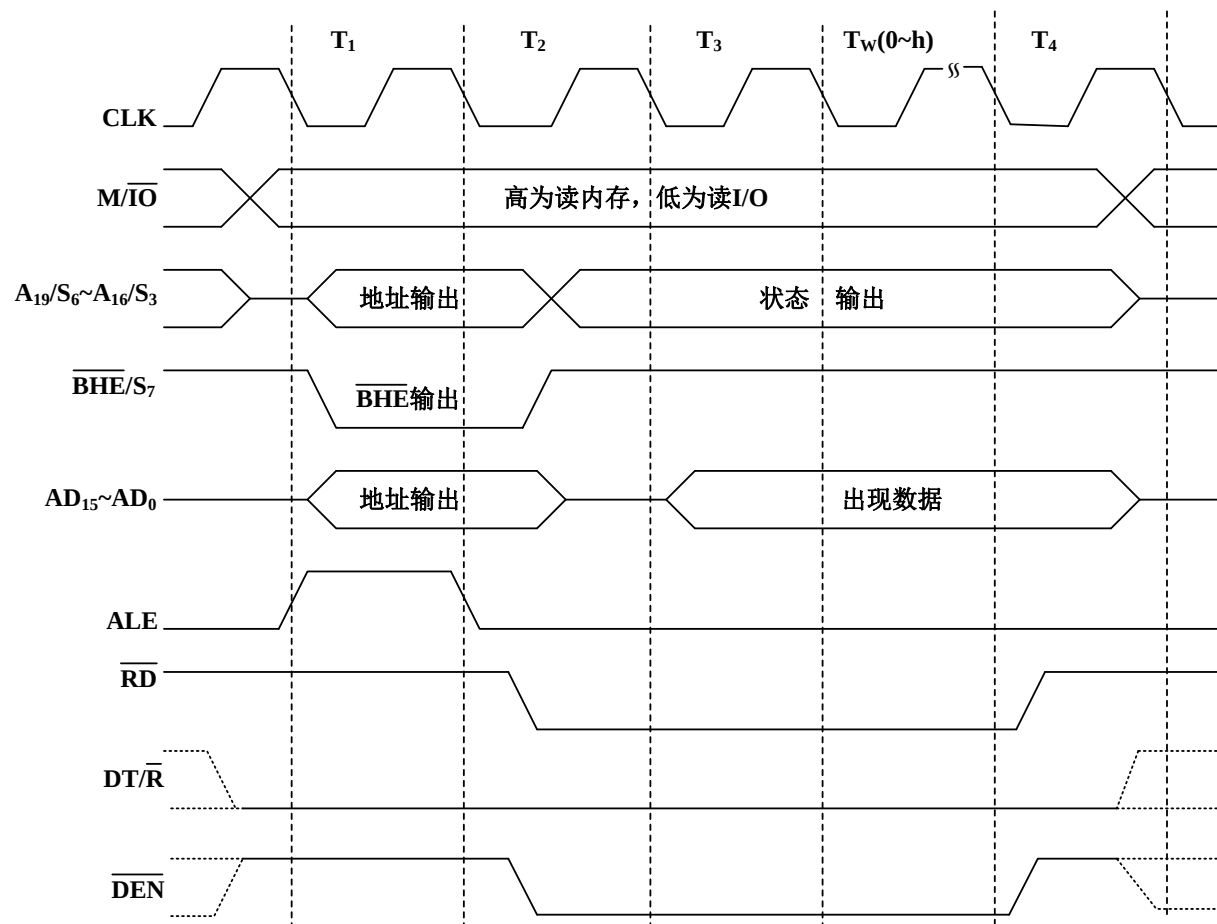
3、时钟周期

- 每条指令的执行包括取指令（**fetch**）、译码（**decode**）和执行（**execute**）。
- 指令周期是指执行一条指令所用的时间。
- 指令周期是由1个或多个总线周期组合而成。或者说，指令周期可以被划分为若干个总线周期。
- **8086**中不同指令的指令周期是不等长的。
- 对于**8086CPU**来说，在**EU**执行指令的时候，**BIU**可以取下一条指令。由于**EU**和**BIU**可以并行工作，**8086**指令的最短执行时间可以是两个总线周期，一般的加、减、比较、逻辑操作是几十个时钟周期，最长的为**16**位乘法约要**200**个时钟周期。

三者关系：

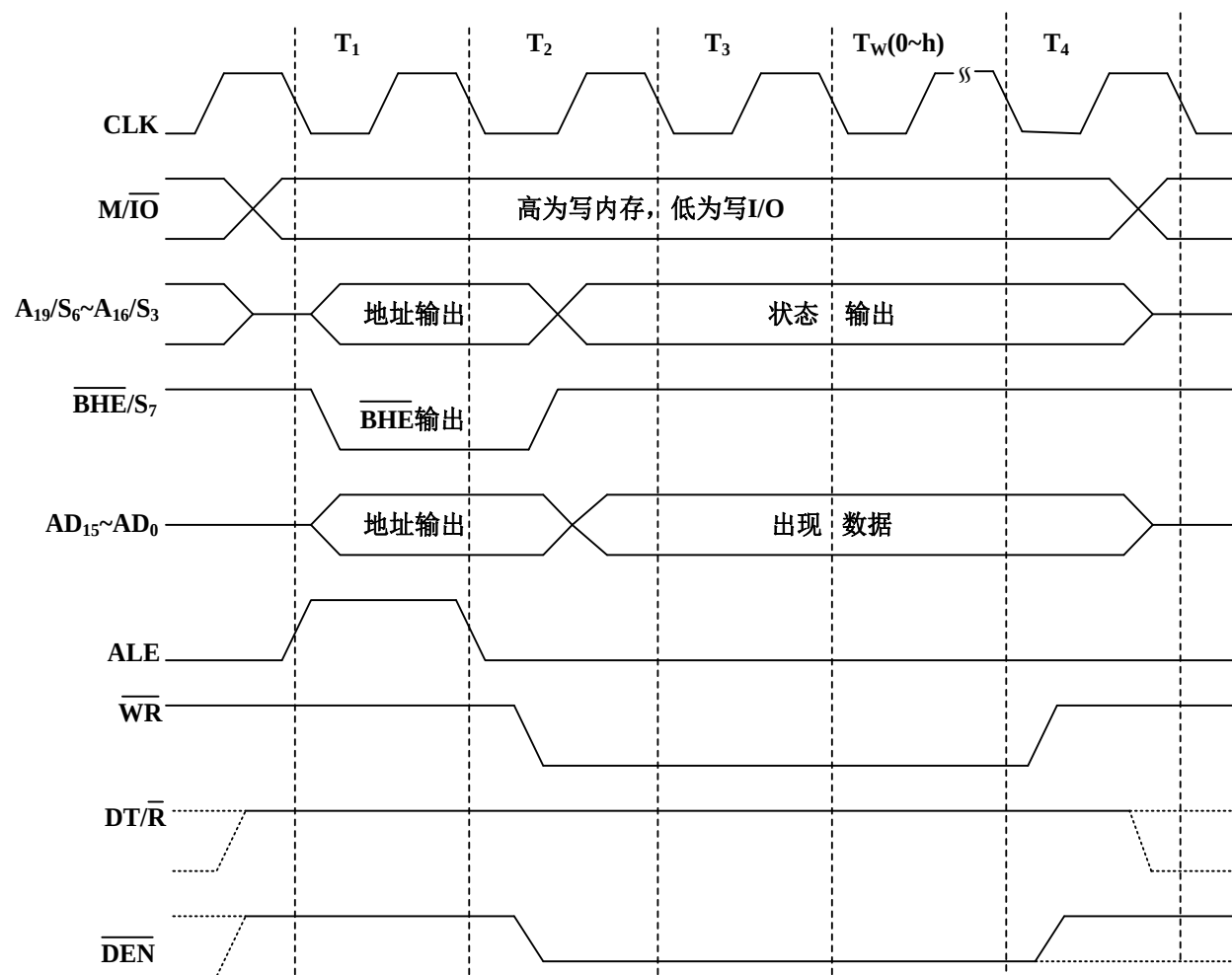


2.4.2 最小模式下的总线周期时序



1. 最小模式下的读周期时序
- 读周期是指CPU从存储器或I/O端口读取数据的总线周期。

2.4.2 最小模式下的总线周期时序



2. 最小模式下的写周期时序

■ 写周期是指CPU中的数据送到存储器或I/O端口的总线周期。

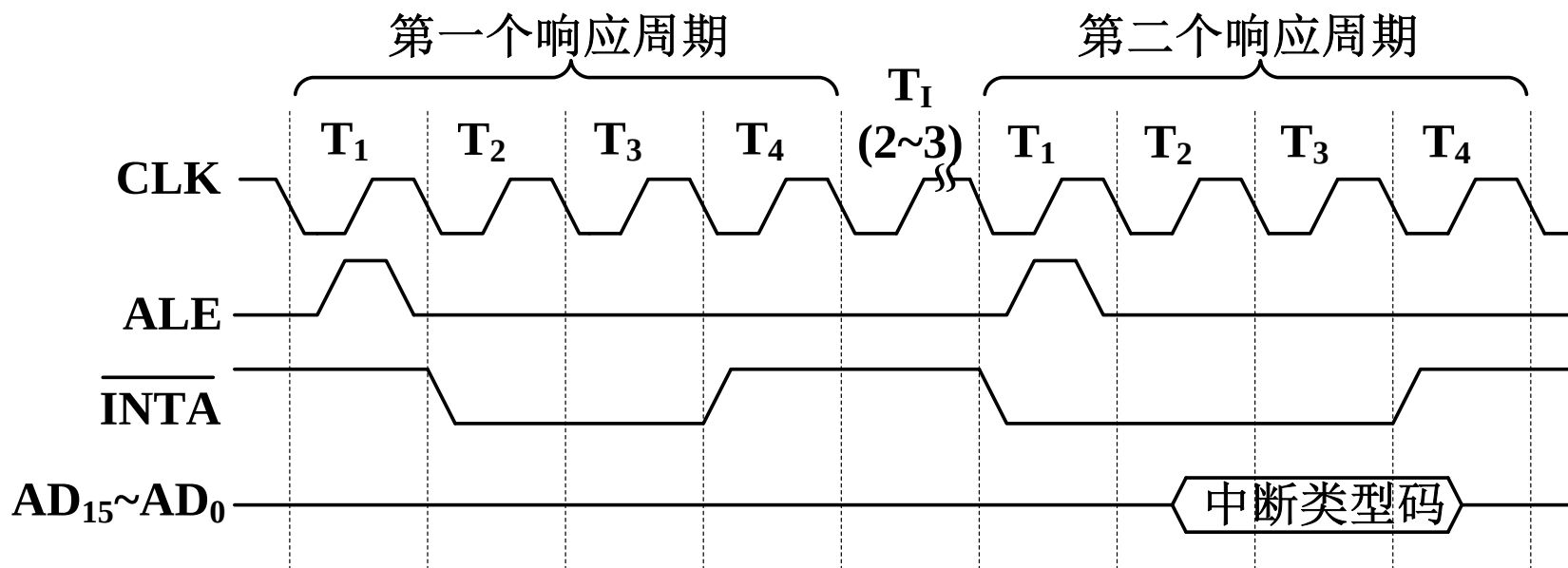
3.4.2 最小模式下的总线周期时序

3. 中断响应周期时序

- 中断是指**CPU**中止当前程序的执行，而去执行一个中断服务程序的过程。
- **8086**可以处理**256**种不同的中断，分为硬件中断和软件中断。硬件中断是由外部设备引发的，分为非屏蔽中断和可屏蔽中断两类。
- 非屏蔽中断是通过**CPU**的**NMI**引脚引入的，其请求信号为一上升沿触发信号，并要求维持两个时钟周期的高电平。当有非屏蔽中断请求时，不管**CPU**正在做什么事情，都会响应，级别很高。

3.4.2 最小模式下的总线周期时序

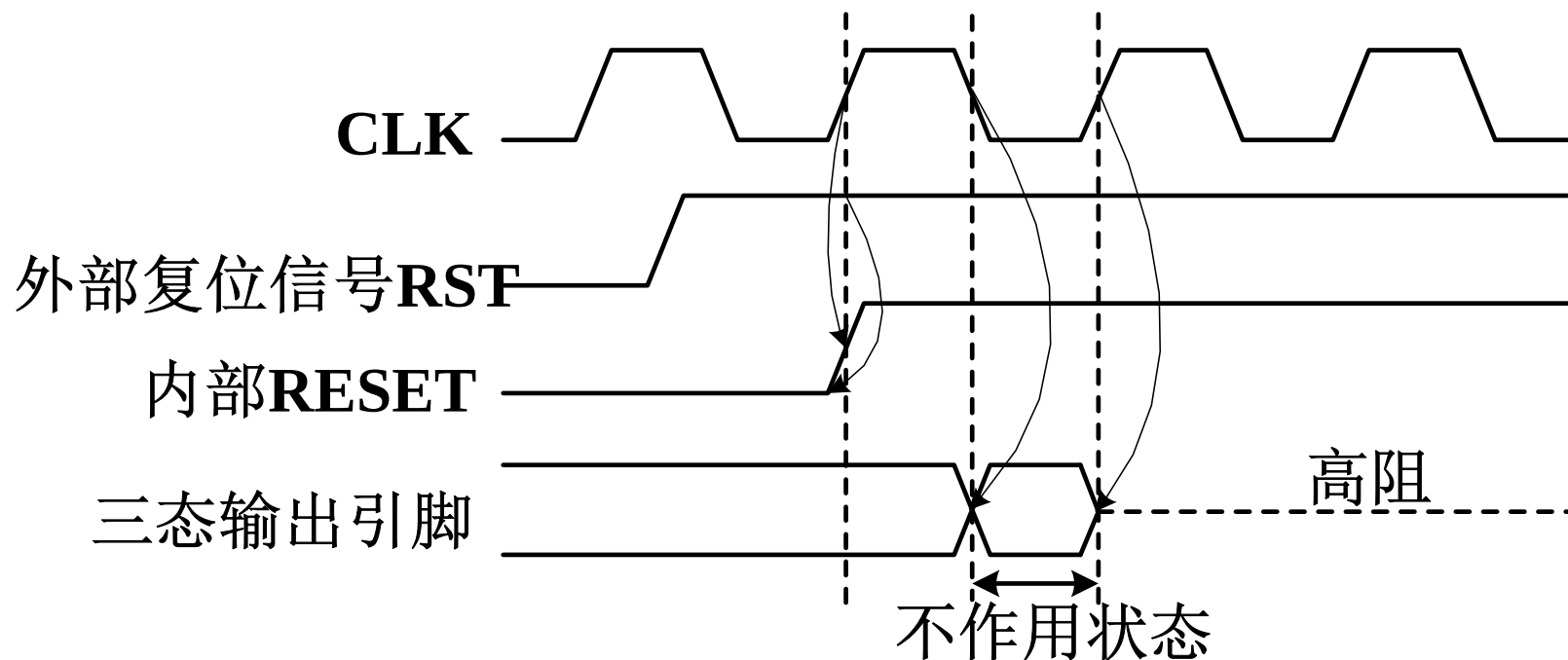
- 一般外部设备的中断是通过**INTR**引脚向**CPU**发出中断请求的，这个可屏蔽中断请求信号的有效电平（高电平），必须维持到**CPU**响应中断为止。若标志**IF=1**，表示**CPU**允许中断，此时**CPU**在执行完当前指令后响应中断，其中断响应周期时序如下图所示。



3.4.2 最小模式下的总线周期时序

4. 8086的复位时序（对两种模式都一样）

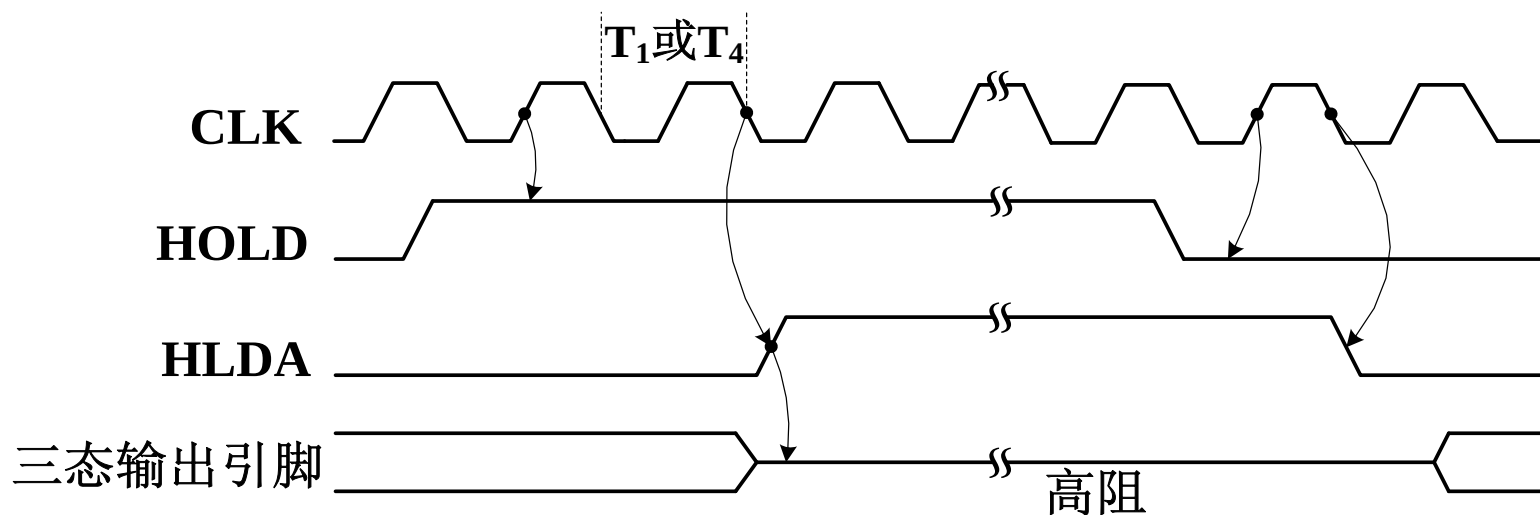
- **RESET**引脚是用来启动或重新启动系统的。外部来的复位信号**RST**经**8284**同步为内部**RESET**信号，如下图所示。



3.4.2 最小模式下的总线周期时序

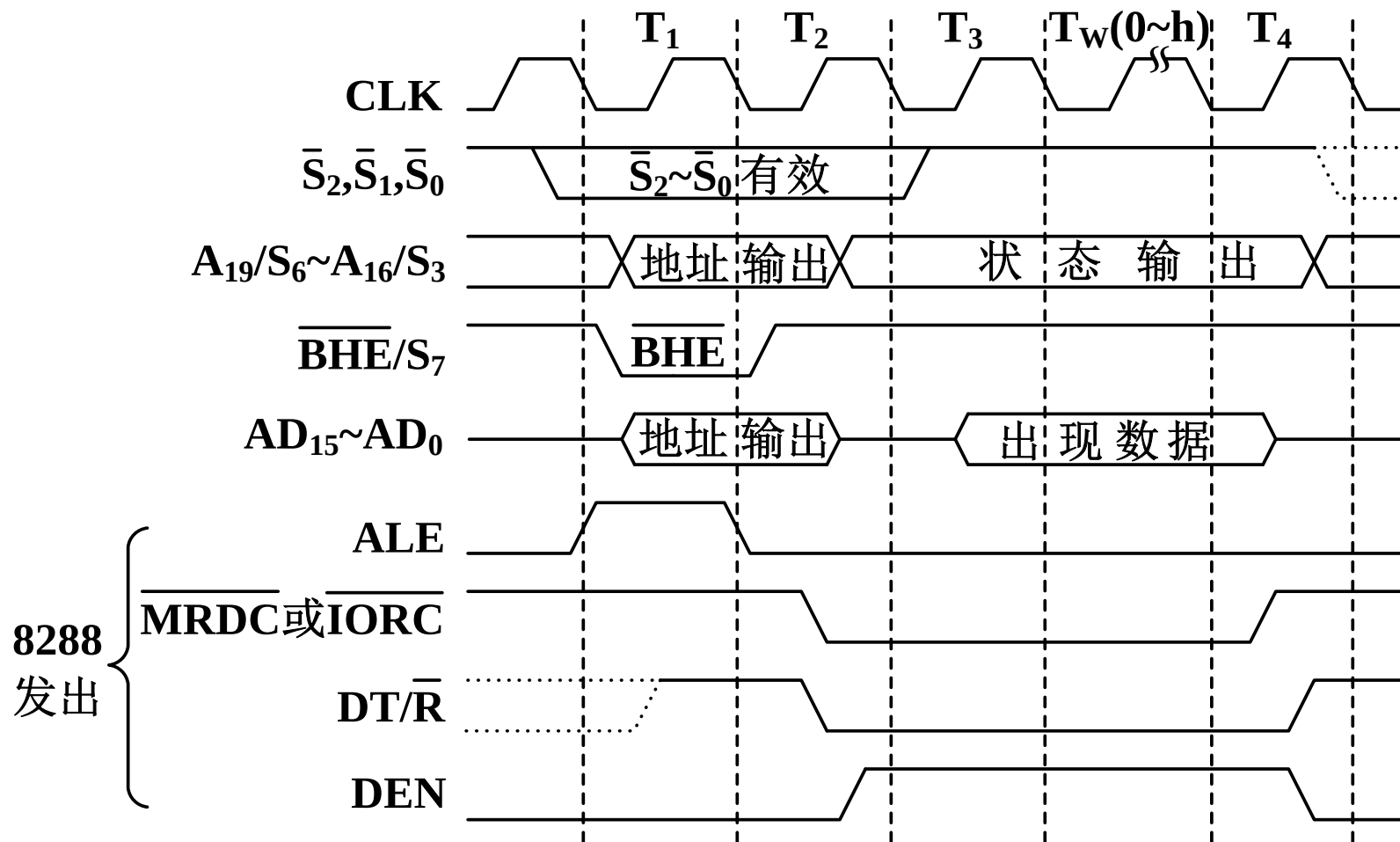
5. 总线保持请求与响应周期时序

- 当系统中**CPU**之外的总线主设备需要占用总线时，就向**CPU**发出一个有效的总线保持请求信号**HOLD**，这个**HOLD**信号可能与时钟信号不同步。**CPU**在每个时钟周期的上升沿进行检测，当检测到该信号时，在当前总线周期的**T4**后或下一个总线周期的**T1**后，**CPU**发出一个有效的保持响应信号**HLDA**，并让出总线，此时，进入了**DMA**传送过程。当**DMA**传送结束后，发出总线保持请求的设备将**HOLD**恢复为低电平，**CPU**才收回总线控制权。这个过程称为总线保持请求与响应周期，如下图所示。



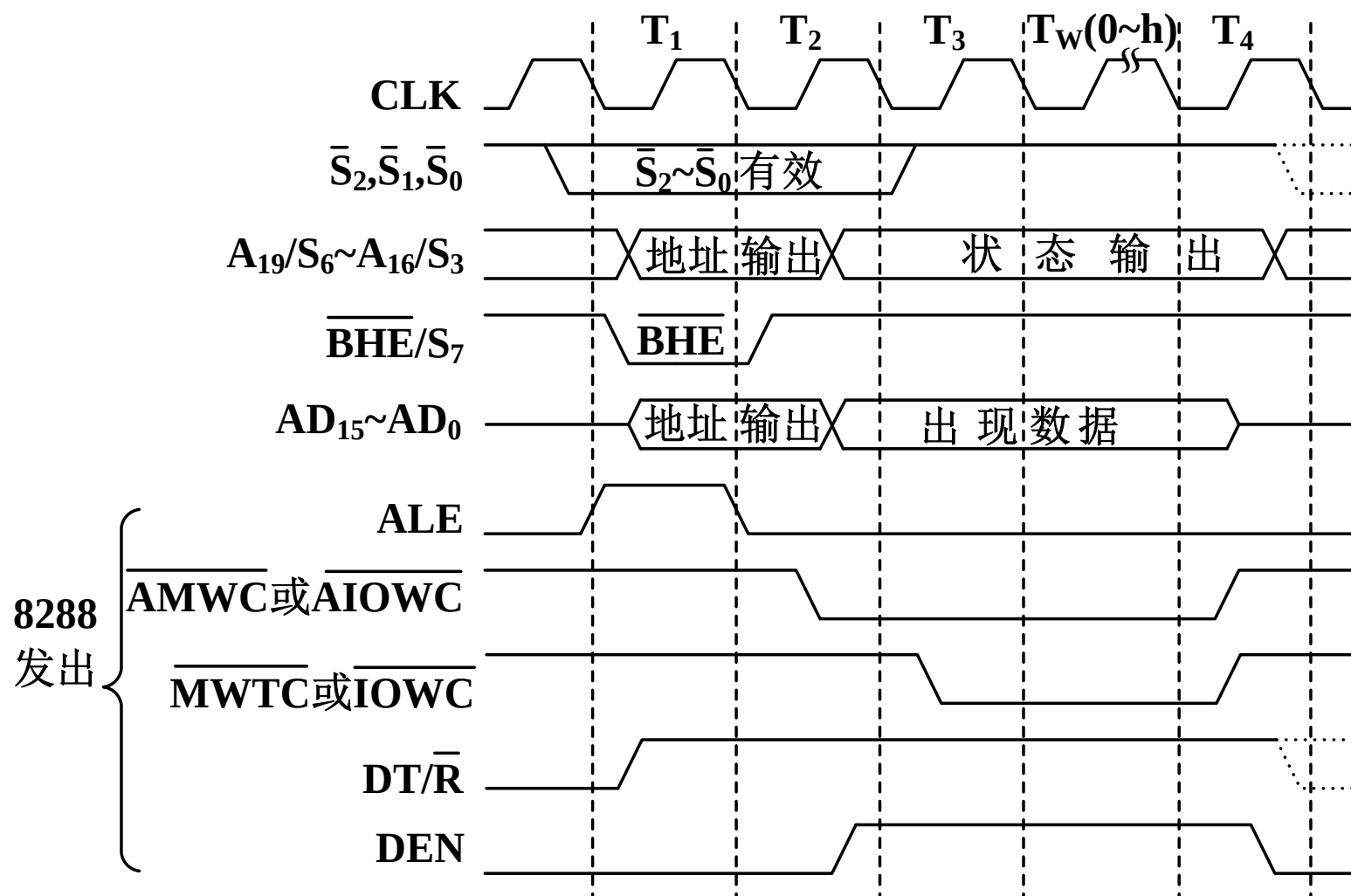
2.4.3 最大模式下的总线周期时序

1. 最大模式下的读周期时序



2.4.3 最大模式下的总线周期时序

2. 最大模式下的写周期时序



2.4.3 最大模式下的总线周期时序

3. 最大模式下的总线请求/允许/释放操作

- **8086**在最大模式下提供的总线控制联络信号是具有双向传输信号的引脚 $\overline{RQ}/\overline{GT}$ ，称之为总线请求/总线允许/总线释放信号。它们可以分别连接到两个其它的总线主模块。在最大模式下，可发出总线请求的总线主模块包括：协处理器和**DMA**控制器等。

